Benchmarking Categorical Clustering Methods Across Diverse Datasets

Mahmoud Alwaraqi, Senior Data Scientist

August 09, 2026

Summary

Categorical data appears across a wide range of diverse industries. In market research and e-commerce, examples include survey data, point-of-sale (POS) data, longitudinal customer transaction data, and subscription preferences. In finance and banking, examples include credit card applications, insurance claim logs, and investment profiles based on risk tolerance, financial goals, etc. Categorical data is also highly relevant in the healthcare and medicine industries where data consists of symptom checklists, patient intake records, and clinical trial profiles. All of these industries, and more, benefit from utilizing clustering methods. The data may consist of nominal (e.g., color) or ordinal variables (e.g., income brackets) and exclude continuous variables from the analysis to focus the scope of this paper on categorical data. The framework of some of the methods mentioned here can easily extend to continuous variables, even without recoding them, such as Finite Mixutre Models (FMMs).

In this paper I compare clustering methods on different categorical datasets. The datasets I experiment with in this paper are the following:

- Iris
- Mushroom
- Zoo

The methods I use for clustering categorical data include the following: K-Modes, Hierarchical clustering, and Finite Mixture Models (FMMs) including Latent Class Models (LCMs).

This paper demonstrates the advantages of Finite Mixture Models as they tend to give consistent quality performance and outperform other methods.

Overview of the methods

K-Modes

k-modes clustering is an extension of k-means clustering, where the mode is used to define the center of the cluster instead of the mean. Recall that the mode is defined as the most frequently occurring value. It's useful when working with categorical data but can be relatively more computationally intensive, especially as the number of variables increases.

The steps are the following [Malik and Tuckfield, 2019]

1. Choose any k number of random points as cluster centers.

2. Find the Hamming distance of each point from the center.
3. Assign each point to a cluster whose center it is closest to.
4. Choose new cluster centers in each cluster by finding the mode of all data points in that cluster.
5. Repeat from step 2 until the cluster centers stop changing.

To pick the number of clusters we will use the average silhouette scores/widths. For each data point i with features $y_i \in C_I$ the silhouette is

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}, \text{ if } |C_I| > 1$$

where

$$a(i) = \frac{1}{|C_I| - 1} \sum_{y_k \in C_I, k \neq i} d(y_i, y_k)$$

$$b(i) = \min_{J \neq I} \frac{1}{|C_J|} \sum_{y_j \in C_J} d(y_i, y_j)$$

$a(i)$ is the average distance of point i to other points in its cluster. $b(i)$ is the minimum of the set of average distances between point i and other other points in different clusters. $s(i)$ ranges between -1 and 1, higher values mean the observation is well matched to its cluster. Given any number of clusters, k, the average of $s(i)$ across all observations will be an indicator of whether or not k is good. A value above 0.70 is considered best, reasonable above 0.50, and weak above 0.25. we will use the `klaR` package to run k-modes on our data.

Hierarchical clustering (HC)

Hierarchical clustering is a tree-based method. There are two flavors of HC:

1. Agglomerative, which starts with all data points assigned their own clusters, and
2. Divisive, which starts with all data points assigned to a single cluster

HC is similar to k-modes in that we must decide on the number of clusters based on silhouettes, the elbow method, or visual inspection of the dendrogram. However, since we're using using categorical data we need to decide on a measure of distance which we take to be Gower's distance with weights set to 1, which is simply the total number of mismatches.

$$d(i, j) = \frac{\sum_{k=1}^{n} w_k \cdot \delta_{ijk} \cdot d_{ijk}}{\sum_{k=1}^{n} w_k}$$

Where:

- n is the total number of features/variables in the dataset.
- w_k is the weight assigned to feature k (typically set to 1 by default).
- δ_{ijk} is a binary indicator (0 or 1) that drops to 0 if either observation has a missing value (`NA`) for feature k, effectively ignoring it.
- d_{ijk} is the specific partial distance calculated for feature k based on its specific data type.

However, as we move into higher dimensional data, Gower's distance fails and it's likely we'd need to resort to Multiple Correspondence Analysis (MCA) before running HC, this method is known as HCPC, or hierarchical clustering on principal components.

Finite Mixture Models (FMMs)

Probability-based cluster modeling has several advantages, as argued by Bouveyron et al. [2019]. Although I will not list those advantages here, I'll say that probability-based clustering seems to perform comparably well to, and in many cases better than, non-probability models. Broadly, the FMMs is the following:

$$f(x) = \sum_{k=1}^{K} \pi_k f_k(x)$$

where

- π_k represents the **mixing proportions** (the probability that a randomly selected observation belongs to cluster k).
- $f_k(x)$ represents the **component density** (the probability distribution specific to cluster k).

As with other clustering methods, in an unsupervised learning context, the total number of clusters, K, will need to be estimated.

All nominal variables

When the categorical variables are all nominal, we use Latent Class Models (LCMs) as formulated in the `Rmixmod` R package. Latent class analysis was first introduced by the Mathematical Sociologist Paul F. Lazarsfeld to analyze survey response patterns.

The set of responses from observation i with d categorical variables is denoted by y_i, each variable with, potentially, a different number of levels. y_i is coded by the binary vector $\left(y_i^{jh}; j = 1, ..., d; h = 1, ..., m_j\right)$ with

$$y_i^{jh} = \begin{cases} 1 & \text{if } y_i^j = h \\ 0 & \text{otherwise.} \end{cases}$$

Data are modeled as a mixture of G multivariate multinomial distributions with probability distribution function [Bouveyron et al., 2019]

$$f\left(y_i; \theta\right) = \sum_{g=1}^{G} \tau_g \prod_{j,h} \left(\alpha_g^{jh}\right)^{y_i^{jh}}$$

where α_g^{jh} is the probability that variable j has level h if observation i is in cluster g. And τ_g are the mixing proportions of the G latent clusters.

The log-likelihood is

$$\mathcal{L}(\theta) = \mathcal{L}(\theta; y) = \sum_{i=1}^{n} \log \left(\sum_{g=1}^{G} \tau_g \prod_{j=1}^{d} \prod_{h=1}^{m_j} \left(\alpha_g^{jh}\right)^{y_i^{jh}} \right)$$

Conditioning on the latent *class* (i.e., cluster), $z = (z_1, ..., z_G)$ with $z_g = (z_{1g}, ..., z_{ng})$ and $z_{ig} = 1$ if y_i is from cluster g and $z_{ig} = 0$ otherwise, we get the complete-data log-likelihood

$$\mathcal{L}_C(\theta) = \mathcal{L}(\theta; y, z) = \sum_{i=1}^{n} \sum_{g=1}^{G} z_{ig} \log \left(\tau_g \prod_{j=1}^{d} \prod_{h=1}^{m_j} \left(\alpha_g^{jh}\right)^{y_i^{jh}} \right)$$

The complete-data log-likelihood is then reparameterized to define a family of five parsimonious LCMs based on five constraints on the parameters, as the following[1]:

1. The α-probability depends upon clusters, variables, and variable levels.
2. The α-probability depends upon clusters, variables, and **not** variable levels.
3. The α-probability depends upon clusters, **not** variables.
4. The α-probability depends upon variables, **not** clusters, and **not** variable levels.
5. The α-probability is constant over variables and clusters.

Additionally, the mixing proportions, τ_g, can either be free or constant, with $\tau_g = p$ for all g. This leads to ten different constraints on the parameters. The ICL criterion will be used to select the best model and the number of clusters for it. Bouveyron et al. [2019] favors ICL for clustering and BIC for density estimation.

Nominal and/or all ordinal

We use a latent threshold model formulation as implemented in the the `clustMD` R package.

The package handles ordinal variables by projecting them into an unobserved, continuous **latent space**. Let $\mathbf{z}_i = (z_{i1}, \ldots, z_{iJ})'$ represent the hidden continuous vector for observation i across J dimensions. If observation i is assigned to latent cluster g, its distribution is modeled as a multivariate Gaussian density:

$$\mathbf{z}_i \mid \text{Cluster } g \sim \text{MVN}(\boldsymbol{\mu}_g, \boldsymbol{\Sigma}_g)$$

To maintain computational feasibility during Monte Carlo EM optimization, the package restricts the latent covariance matrix $\boldsymbol{\Sigma}_g$ to a diagonal structure:

$$\boldsymbol{\Sigma}_g = \text{diag}(\sigma^2_{g1}, \sigma^2_{g2}, \ldots, \sigma^2_{gJ})$$

This means that within any single cluster, the latent variables are assumed to be independent, while their overall geometry is modified across groups according to the chosen `model` argument:

- **`model = "EII"` (Spherical, Equal Volume):** Constrains the covariance to:

  $$\boldsymbol{\Sigma}_g = \sigma^2 \mathbf{I}_J$$

  All variables share an identical variance, forcing clusters into rigid, circular shapes of equal volume across all groups.

- **`model = "VII"` (Spherical, Variable Volume):** Constrains the covariance to:

  $$\boldsymbol{\Sigma}_g = \sigma^2_g \mathbf{I}_J$$

  Variables within a cluster share the same variance, but the total volume (the scale of σ^2_g) can vary freely between different clusters.

[1] Technically, by "α-probability" in the list of these five models I mean specifically the "scatter" parameter that results from the reparameterization of the complete-data log-likelihood which is not shown in this article. Essentially, the "scatter" is a reparameterizaiton of the probability α^{jh}_g that reflects how the probabilities of the levels of a variable could differ from the probability of the mode value of said variable. If you'd like more details please see Bouveyron et al. [2019].

- `model = "EEI"` **(Diagonal, Equal Volume & Shape):** Constrains the covariance to:

$$\mathbf{\Sigma}_g = \mathbf{\Sigma} = \text{diag}(\sigma_1^2, \sigma_2^2, \ldots, \sigma_J^2)$$

  Variables can exhibit unique variances from one another (producing elliptical clusters), but this exact shape and volume profile is held constant across all groups.

- `model = "VVI"` **(Diagonal, Variable Volume & Shape):** Unrestricted diagonal covariance given by:

$$\mathbf{\Sigma}_g = \text{diag}(\sigma_{g1}^2, \sigma_{g2}^2, \ldots, \sigma_{gJ}^2)$$

  This is the most flexible model, allowing every single variable in every individual cluster to scale its variance independently, yielding unique elliptical shapes and sizes for each group.

Measuring performance

I use the **homogeneity score** to measure model performance. It is an information-theoretic cluster validation metric used to evaluate unsupervised machine learning models; specifically for this paper, predicted clusters against the true classes. A model satisfies perfect homogeneity ($h = 1.0$) if all of its predicted clusters contain only data points that are members of a single class.

The metric ignores over-clustering; it does not penalize a model for splitting a true class into multiple, highly pure sub-clusters. For example, our true class may be binary: customer purchased or not. However, the clustering model identifies patterns such as cluster 1 = "18 to 24 year old's who browsed the online gaming section for more than 10 minutes" as a cluster. Say there are 15 such patterns/subgroups that the clustering model identities. We are concerned with the purity/homogeneity of these clusters; that is, being entirely, or mostly, "purchased" or "not purchased". If individuals with attributes in cluster 1 is close to 100% or 0% purchased then that is a good thing; what we don't want is something like 50% purchased in cluster 1. We are **not** concerned with, and will not penalize, the clustering model for finding 15 of these patterns/subgroups instead of only the 2, purchased vs not-purchased. What we care about is how well each cluster identifies with a single class.

Mathematical Formulation

The formulation relies entirely on Shannon Entropy and Conditional Entropy to measure how much uncertainty about the true class labels (C) is removed given the predicted cluster assignments (K).

The Core Formula

The formal mathematical definition of the homogeneity score (h) is:

$$h = 1 - \frac{H(C \mid K)}{H(C)}$$

Where: * $H(C)$ is the baseline entropy of the ground-truth classes. * $H(C \mid K)$ is the conditional entropy of the classes given the predicted clusters.

True Class Baseline Entropy $H(C)$

This measures the inherent uncertainty or structural diversity of the true classes across the entire dataset:

$$H(C) = -\sum_{c=1}^{|C|} \frac{n_c}{n} \log\left(\frac{n_c}{n}\right)$$

where:

- n: Total number of data points in the dataset.
- n_c: Number of data points belonging to true class c.
- $\frac{n_c}{n}$: The probability that a randomly selected point belongs to class c.

Conditional Entropy $H(C \mid K)$

This tracks the remaining uncertainty about the true classes *after* inspecting the contents of the predicted clusters:

$$H(C \mid K) = -\sum_{k=1}^{|K|} \sum_{c=1}^{|C|} \frac{n_{c,k}}{n} \log\left(\frac{n_{c,k}}{n_k}\right)$$

Where:

- n_k: Total number of points assigned to predicted cluster k.
- $n_{c,k}$: Number of points that belong to true class c and were assigned to cluster k.
- $\frac{n_{c,k}}{n_k}$: The proportion of cluster k that belongs to class c (representing cluster purity).

Boundary Behaviors

- **Perfect Purity ($h = 1.0$):** If every cluster contains points from only one true class, then $\frac{n_{c,k}}{n_k} = 1$ for that class and 0 for all others. Because $\log(1) = 0$, the conditional entropy drops to zero ($H(C \mid K) = 0$), yielding $h = 1 - 0 = 1.0$.
- **Random Clustering ($h \approx 0.0$):** If a cluster assignment provides zero structural information, the distribution of classes inside each cluster mirrors the overall dataset. Thus, $H(C \mid K) = H(C)$, yielding $h = 1 - 1 = 0.0$.

Iris dataset

The Iris dataset is a classic data frame introduced by British statistician Ronald Fisher in 1936, it is often used sanity-check basic functionality. It consists of 150 observations split evenly across three distinct species of Iris flowers, Setosa, Versicolor, and Virginica. It comprises samples of four measurements: Sepal.Length, Sepal.Width, Petal.Length, and Petal.Width. For testing, we can pretend we don't know the species and use the four measurements to guess it. Since we're interested in clustering with categorical variables, let's recode the four numeric measurement variables. variables.

Table 1: Iris percentiles

	5%	25%	50%	75%	95%
Sepal.Length	4.6	5.1	5.8	6.4	7.3
Sepal.Width	2.3	2.8	3.0	3.3	3.8
Petal.Length	1.3	1.6	4.3	5.1	6.1
Petal.Width	0.2	0.3	1.3	1.8	2.3

We will create four levels for each variable based on the values of the quartiles.

The data is recoded to “Low”,“Medium”,“Medium-High”,“High”.

Table 2: Counts of four iris categorical variables

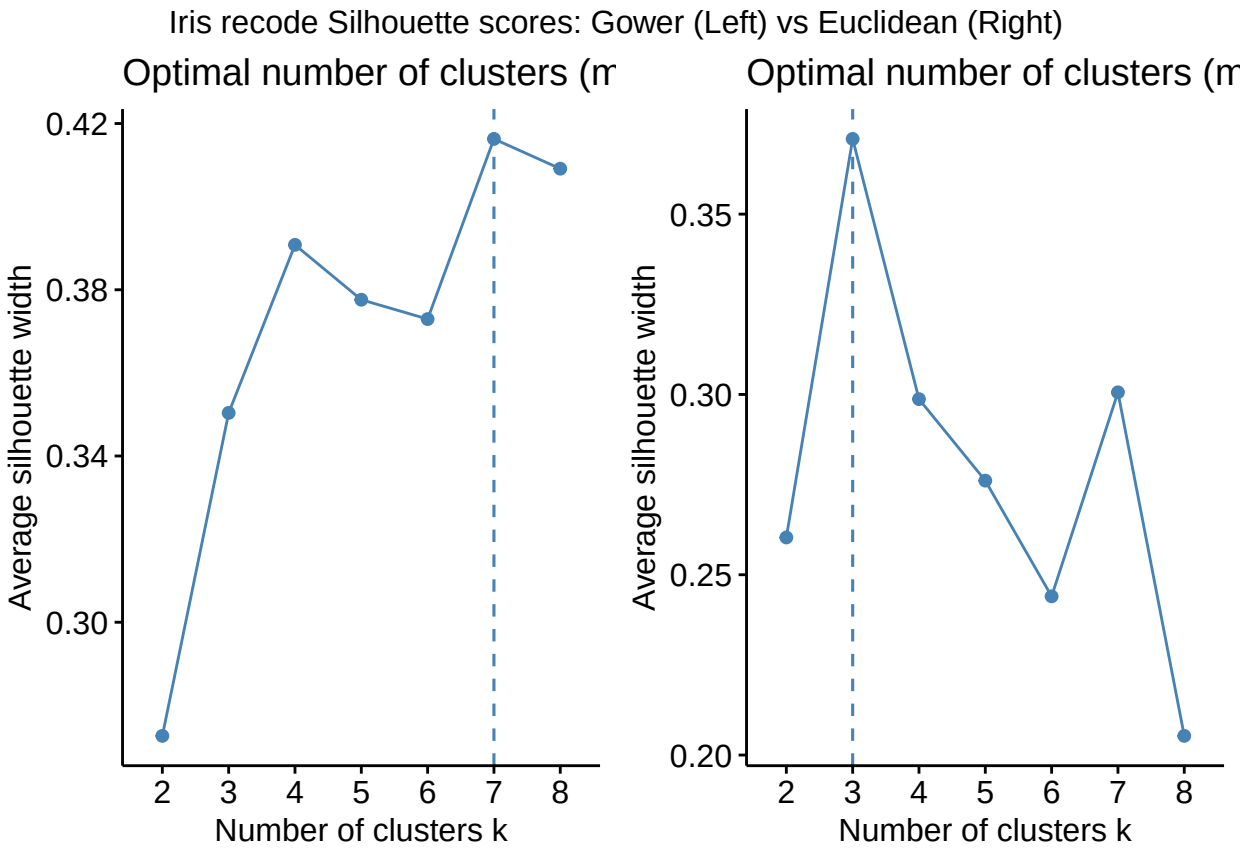
Recode Variable	Low	Medium	Medium-High	High
sepal_length	32	41	35	42
sepal_width	33	24	50	43
petal_length	37	38	33	42
petal_width	34	31	39	46

Iris: k-modes

We use `factoextra::fviz_nbclust()` to get silhouette scores for 8 clusters. This function returns a plot of average silhouette scores by cluster, among other things.

If we ignore the ordinal nature of the measurement variables, which we can do by specifying the Gower distance in the `diss` argument of `factoextra::fviz_nbclust()`, then k-modes fails badly.

```
gower_dist <- cluster::daisy(iris_recode, metric = "gower")
set.seed(45)
iris_k_modes_silhouette_nom <- factoextra::fviz_nbclust(
  iris_recode,
  FUNcluster = klaR::kmodes,
  method = "silhouette",
  diss = gower_dist,
  k.max = 8
  )

iris_recode_integers <- iris_recode %>%
    dplyr::mutate(dplyr::across(dplyr::everything(),as.integer))
set.seed(45)
iris_k_modes_silhouette_ord <- factoextra::fviz_nbclust(
  iris_recode_integers,
  FUNcluster = klaR::kmodes,
  method = "silhouette",
  k.max = 8
  )
```



Treating the measurements naturally, as ordinal variables, suggests 3 clusters, while ignoring the ordinal nature of the measurements suggests more than twice the number of clusters.

Know that passing the data as integers or as factors into `klaR::kmodes()` doesn't really matter in general, and certainly not for us here, since the function uses the Hamming distance which is based on matches and mismatches of values.

```
## Optimal number of clusters if treating as nominal
set.seed(45)
iriscat_kmodes_nom_45 <- klaR::kmodes(
  iris_recode,7,iter.max = 1000
  )
## Optimal number of clusters if treating as ordinal
set.seed(45)
iriscat_kmodes_ord_45 <- klaR::kmodes(
  iris_recode,3,iter.max = 1000
  )
```

Table 3: Iris: Methods performance

method_name	variables	parameters	rand.seed	homo_score
k-modes	nominal	k=7	45	0.77
k-modes	ordinal	k=3	45	0.62

Since k-modes, like k-means, can be sensitive to the (random) starting point, we will try a different (random) starting point, `set.seed(323)`.

Table 4: Iris recode: 3-cluster k-modes starting at `set.seed(323)`.

	1	2	3
setosa	24	0	26
versicolor	21	29	0
virginica	8	24	18

Table 5: Iris recode: 7-cluster k-modes starting at `set.seed(323)`.

	1	2	3	4	5	6	7
setosa	5	0	12	0	17	0	16
versicolor	18	23	0	9	0	0	0
virginica	1	8	0	2	0	39	0

Table 6: Iris: Methods performance

method_name	variables	parameters	rand.seed	homo_score
k-modes	nominal	k=7	45	0.77
k-modes	ordinal	k=3	45	0.62
k-modes	ordinal	k=3	323	0.27
k-modes	nominal	k=7	323	0.76

This leads to an important question, what level of performance should we expect? We use bagging to provide a better estimate, using 7 cluster model that we got when ignoring the ordinal nature of the variables.

```
set.seed(323)
num_bootstrap <- 100
num_clusters  <- 7

boot_models <- lapply(1:num_bootstrap, function(i,x = iris_recode) {
  # Sample row indices with replacement (Bagging)
  boot_idx  <- sample(1:nrow(x), replace = TRUE)
  boot_data <- x[boot_idx, ]

  # Fit k-modes on bootstrap data
  km <- klaR::kmodes(boot_data, modes = num_clusters)

  # Predict back on the original data layout and cast to a CLUE partition
  # This standardizes the labels so they can be averaged

  predicted_vector <- predict_kmodes(km, x)

  partition_object <- as.cl_partition(predicted_vector)

  return(partition_object)

})

# Bind predictions into a single Cluster Ensemble object
```

```r
ensemble_bag <- clue::cl_ensemble(list = boot_models)

# Extract Consensus via soft/hard averaging (Default: Euclidean consensus)
ensemble_bag_consensus <- clue::cl_consensus(ensemble_bag)

# Extract final hard cluster assignments
iris_kmodes_bagged_preds <- clue::cl_class_ids(ensemble_bag_consensus)
```

Table 7: Bagged 7 cluster 7 modes: truth vs predicted

	1	2	3	4	5	6	7
setosa	0	21	0	22	0	0	7
versicolor	0	0	10	0	19	0	21
virginica	35	0	13	0	0	2	0

Table 8: Iris: Methods performance

method_name	variables	parameters	rand.seed	homo_score
k-modes	nominal	k=7	45	0.77
k-modes	ordinal	k=3	45	0.62
k-modes	ordinal	k=3	323	0.27
k-modes	nominal	k=7	323	0.76
bagged k-modes	nominal	k=7,iter=100	323	0.81

So starting from a "bad seed" bagging ensured better performance relative to any single k-mode model.

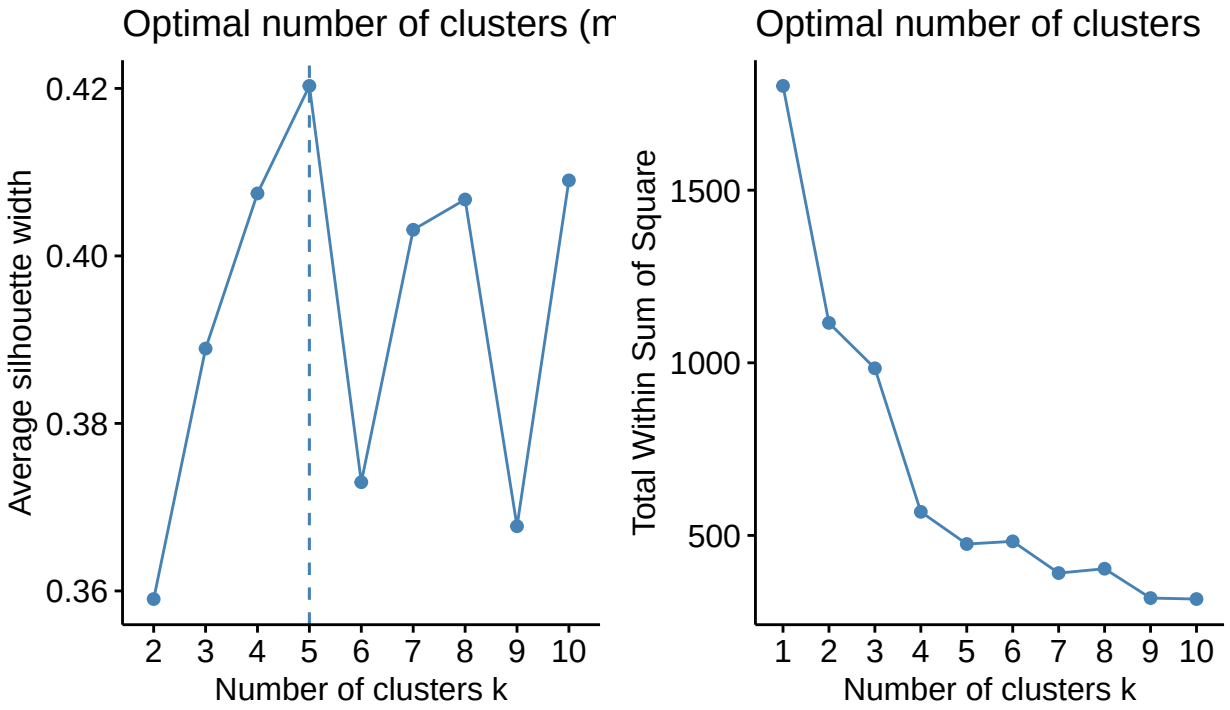
Iris: HC

Let's start with agglomerative hierarchical clustering.

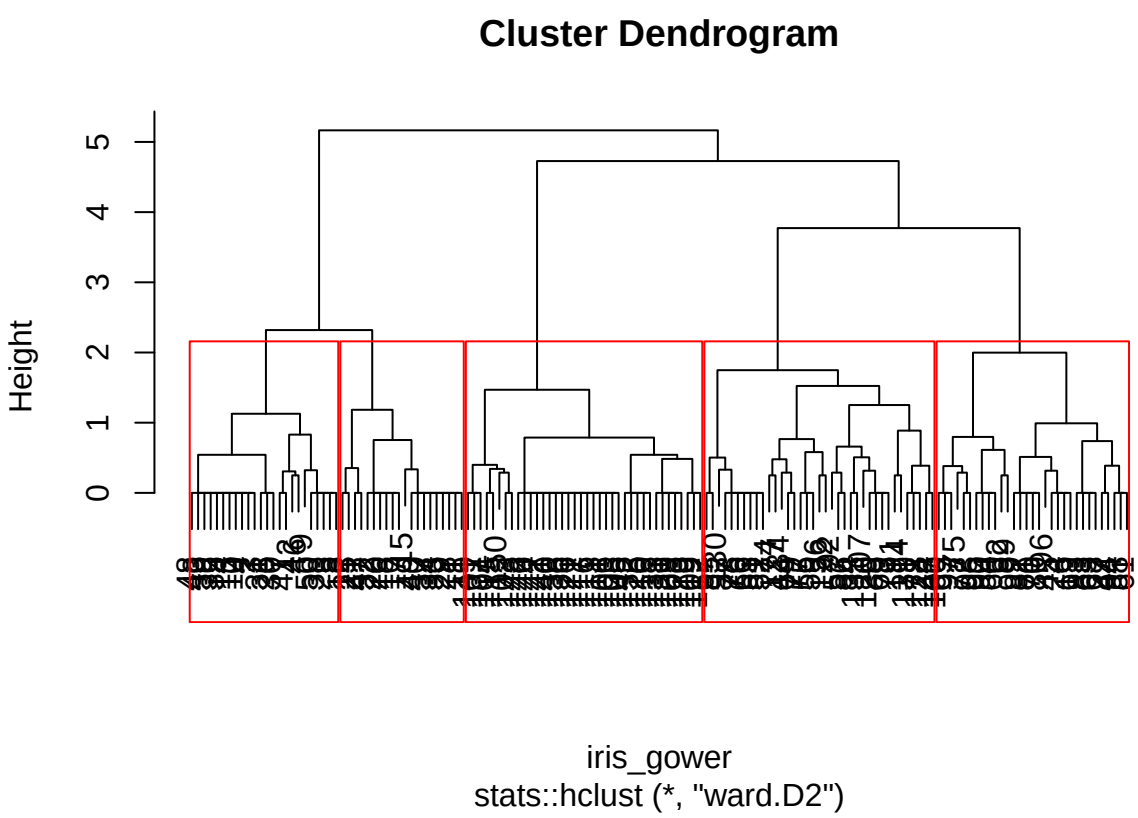
```r
iris_gower <- cluster::daisy(iris_recode,metric = "gower")
set.seed(323)
grid.arrange(
  factoextra::fviz_nbclust(as.matrix(iris_gower), kmeans, method = "silhouette"),
  factoextra::fviz_nbclust(as.matrix(iris_gower), kmeans, method = "wss"),
  ncol = 2,
  top = "Agglomerative Hierarchical Clustering \n
  Iris recode: Gower's Distances with weights 1"
  )
```

Agglomerative Hierarchical Clustering

Iris recode: Gower's Distances with weights 1




```
set.seed(323)
iris_hc <- stats::hclust(iris_gower, method = "ward.D2")
plot(iris_hc)
rect.hclust(iris_hc, k = 5, border = "red")
```



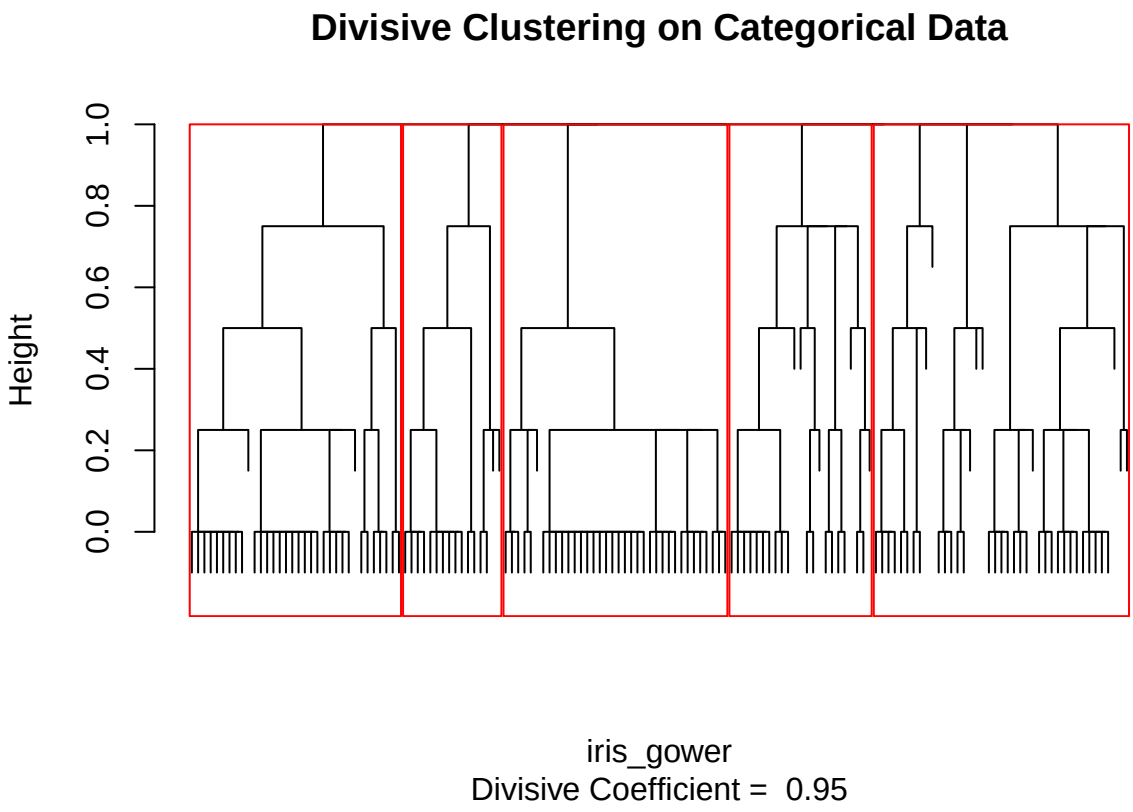

```
iris_hc_classes <- stats::cutree(iris_hc, k = 5)
```

Table 9: Iris: Methods performance

method_name	variables	parameters	rand.seed	homo_score
k-modes	nominal	k=7	45	0.77
k-modes	ordinal	k=3	45	0.62
k-modes	ordinal	k=3	323	0.27
k-modes	nominal	k=7	323	0.76
bagged k-modes	nominal	k=7,iter=100	323	0.81
HC-AGG	nominal	k=5,ward.D2	323	0.77

Now, divisive hierarchical clustering.

```
set.seed(323)
iris_hc_div <- cluster::diana(iris_gower)
plot(
  iris_hc_div, which.plots = 2, labels=FALSE,
  main = "Divisive Clustering on Categorical Data")
rect.hclust(as.hclust(iris_hc_div), k = 5, border = "red")
```




```
iris_hc_div_classes <- stats::cutree(as.hclust(iris_hc_div), k = 5)
```

Table 10: Iris: Methods performance

method_name	variables	parameters	rand.seed	homo_score
k-modes	nominal	k=7	45	0.77
k-modes	ordinal	k=3	45	0.62
k-modes	ordinal	k=3	323	0.27
k-modes	nominal	k=7	323	0.76
bagged k-modes	nominal	k=7,iter=100	323	0.81
HC-AGG	nominal	k=5,ward.D2	323	0.77
HC-DIV	nominal	k=5	323	0.82

Iris: LCM

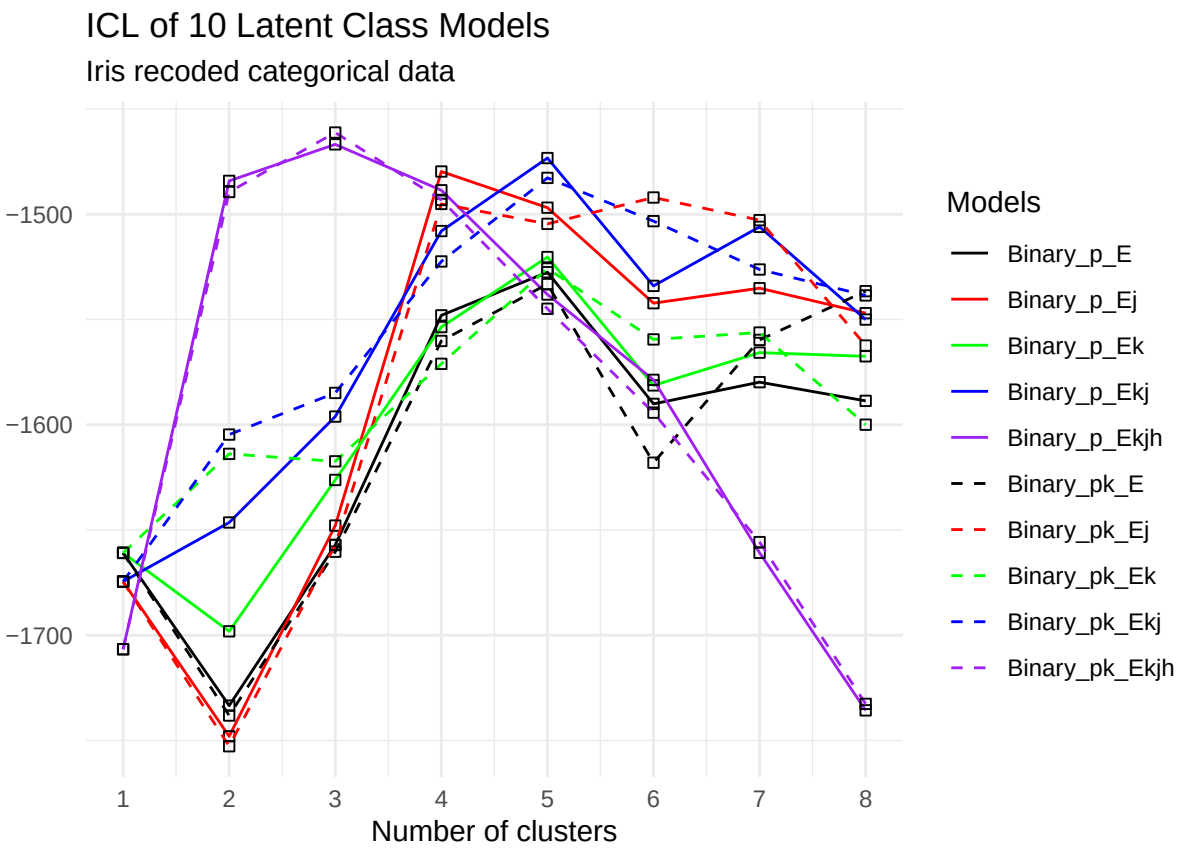
We start by using the ICL criterion to find the best of 10 possible models. We loop over each model using `Rmixmod::mixmodCluster()` and 8 clusters for each.

```
names_possible_models <- c("Binary_p_E","Binary_p_Ej", "Binary_p_Ek",
                           "Binary_p_Ekj","Binary_p_Ekjh","Binary_pk_E",
                           "Binary_pk_Ej","Binary_pk_Ek","Binary_pk_Ekj",
                           "Binary_pk_Ekjh")
```

```
names(names_possible_models) <- names_possible_models
set.seed(323)
iris_lcm_models <-
  purrr::map(names_possible_models,function(x){
    res <- Rmixmod::mixmodCluster(
      iris_recode,
      nbCluster = 1:8,
      dataType = "qualitative",
      model = Rmixmod::mixmodMultinomialModel(listModels = x),
      criterion = c("ICL")
      )
    res
  }
  )
```

Now we get the ICL values by cluster then plot.

```
set.seed(323)
iris_lcm_models_lcl <-
  purrr::map(
    names_possible_models,function(model_name){

      this_model_results <- iris_lcm_models[[model_name]]

      this_model_nbcluster <-
        purrr::map_int(1:8,function(x = .x){
          this_model_results@results[[x]]@nbCluster[1]
        })

      this_model_icl <-
        purrr::map_dbl(1:8,function(x = .x){
          this_model_results@results[[x]]@criterionValue[1]
        })*-1

      dplyr::tibble(num_clusters = this_model_nbcluster,
                    icl = this_model_icl) %>%
        dplyr::rename(!!model_name := icl)
    }


  )
```



The best model is `Binary_p_Ejkh` at 3 clusters. This corresponds to the (parsimonous) LCM with a constant mixing proportion between the 3 clusters (i.e., $\frac{1}{3}$) and with α-probabilities (i.e., technically the scatter) that vary with cluster, variable, and variable level. We pull the results of the best LCM model and compare clusters against iris Species.

```
iris_lcm_models_best <- iris_lcm_models[["Binary_p_Ekjh"]]@bestResult
```

Table 11: Iris species vs. cluster estimated from Binary_p_Ekjh LCM

	1	2	3
setosa	50	0	0
versicolor	0	38	12
virginica	0	4	46

Table 12: Iris: Methods performance

method_name	variables	parameters	rand.seed	homo_score
HC-DIV	nominal	k=5	323	0.82
bagged k-modes	nominal	k=7,iter=100	323	0.81
k-modes	nominal	k=7	45	0.77
HC-AGG	nominal	k=5,ward.D2	323	0.77
k-modes	nominal	k=7	323	0.76

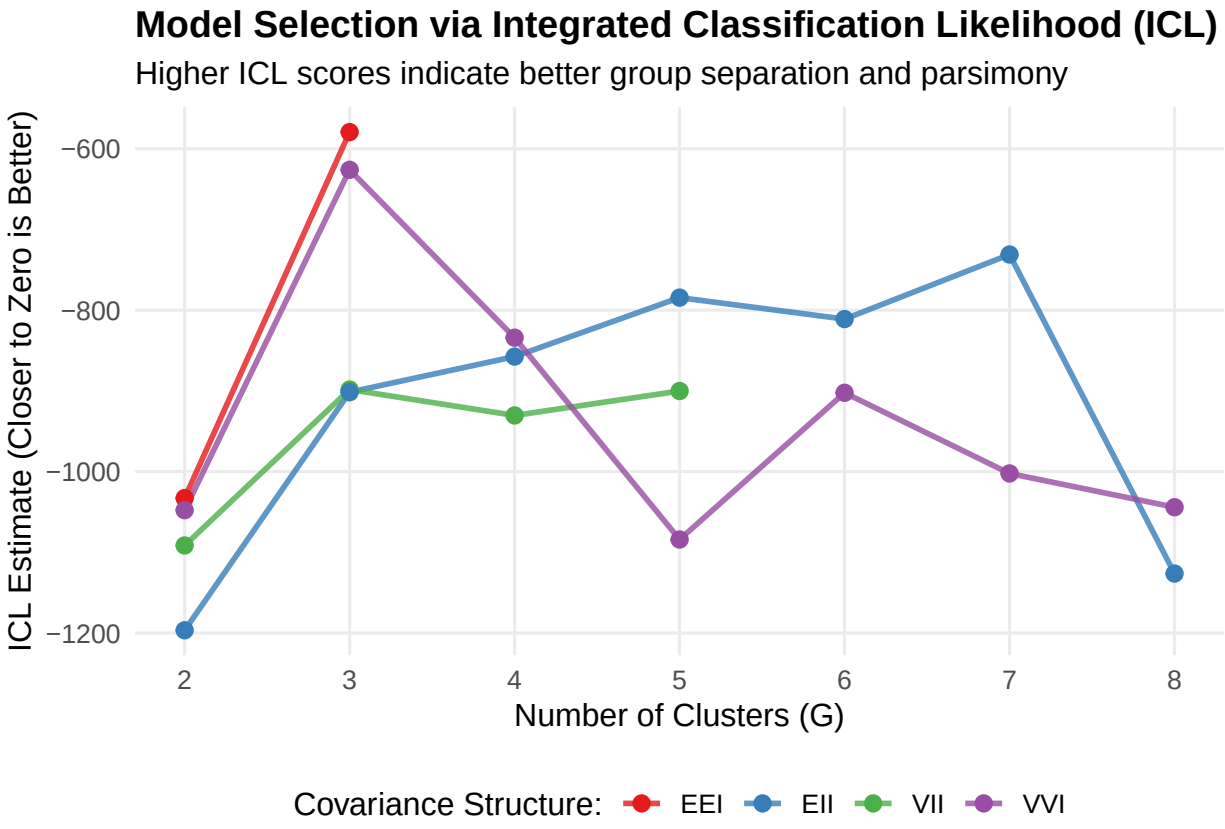
method_name	variables	parameters	rand.seed	homo_score
LCM	nominal	k=3,Binary_p_Ekjh	323	0.74
k-modes	ordinal	k=3	45	0.62
k-modes	ordinal	k=3	323	0.27

Iris: FMM

Now, let's respect the ordinal nature of the measurement variables. The `clustMD` package uses a latent threshold (or latent continuous) framework where observed categorical variables (e.g., ordinal) are assumed to be manifestations of an underlying continuous multivariate Gaussian latent vector. Let's start with model selection.

```
iris_md_matrix <- iris_recode %>%
  dplyr::mutate(across(everything(), ~ as.numeric(
    factor(.,
           levels = c("Low", "Medium", "Medium-High", "High"),
           ordered = TRUE
           )))) %>%
  as.matrix()
# Takes 10 minutes or so to complete
# Can skip if results are already available/saved
if (run_iris_fmm_modelselect){
  # Since we have only nominal/ordinal variables,
  #  we don't need to try all 6 models
  # tested_models <- c("EII","VII","EEI","VVI","VEI","EVI")
  tested_models <- c("EII", "VII", "EEI", "VVI")
  tested_n_clusters <- 2:8

  # Create every unique combination of model type and number of clusters
  search_grid <- base::expand.grid(
    Model = tested_models, G = tested_n_clusters, stringsAsFactors = FALSE)

  # Run the iterative loop over the grid
  set.seed(323)
  model_selection_results <- purrr::map_dfr(1:nrow(search_grid), function(i) {
    current_model <- search_grid$Model[i]
    current_g     <- search_grid$G[i]

    #Safeguard the loop against estimation convergence errors using tryCatch
    fit <- fit_stable_clustMD(
      X = iris_md_matrix,
      G = current_g,
      model_type = current_model
      )

    # If model successfully converged, extract metrics; otherwise return NA
    if (!is.null(fit)) {
      data.frame(
        Model = current_model,
        Clusters = current_g,
        ICL = fit$ICLhat
      )
```

```
    } else {
      data.frame(Model = current_model, Clusters = current_g, ICL = NA)
    }
  })
best_icl <- model_selection_results %>% dplyr::arrange(desc(ICL))
dir.create(file.path(intermediates_dir,"iris","fmm"),recursive = TRUE)
saveRDS(best_icl,file.path(intermediates_dir,"iris","fmm","best_icl.rds"))
} else {
  best_icl <- readRDS(file.path(intermediates_dir,"iris","fmm","best_icl.rds"))
}
best_icl <- best_icl %>%
  dplyr::filter(!is.na(ICL)) %>%
  dplyr::arrange(desc(ICL))
```



Some points not shown because the model didn't converge within the maximum number of iterations that we specified for the given number of clusters. The best FMM model over our grid search is EEI with 3.

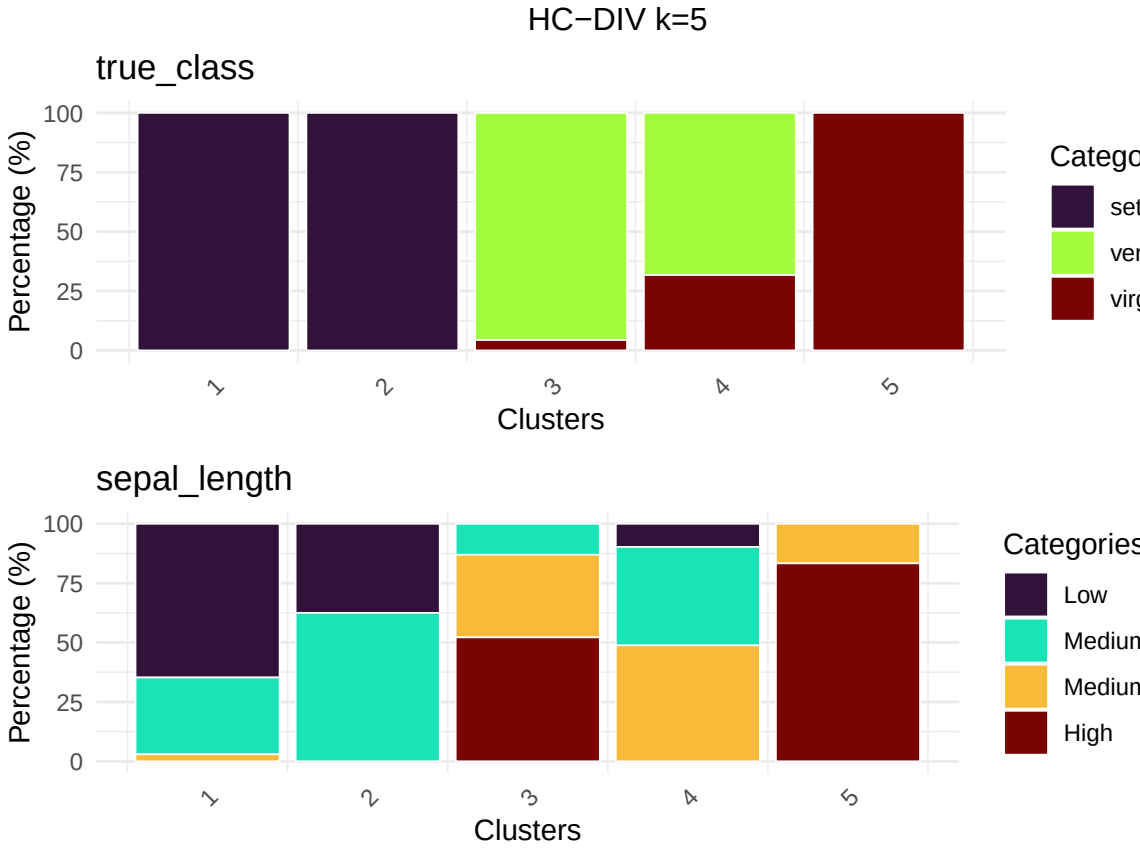
Table 13: Iris: Model Performance

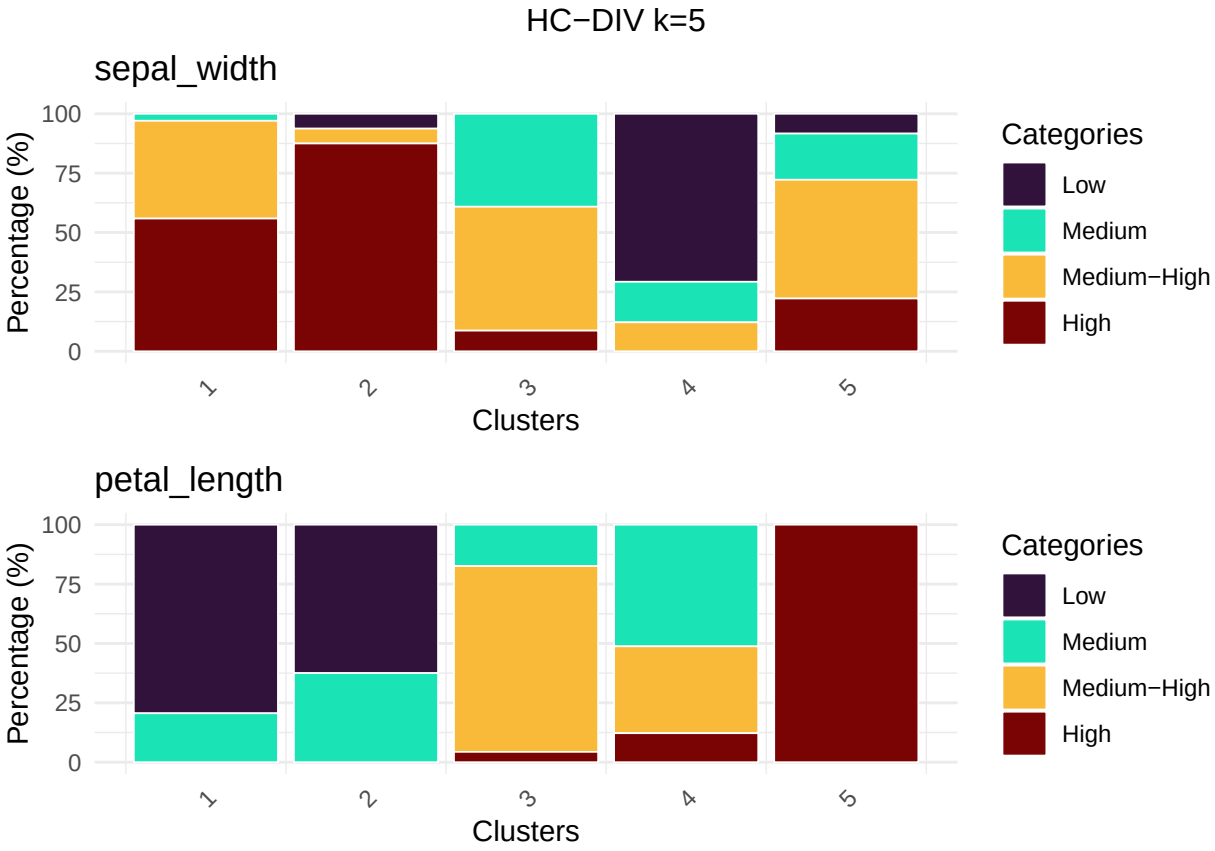
method_name	variables	parameters	rand.seed	homo_score
HC-DIV	nominal	k=5	323	0.82
bagged k-modes	nominal	k=7,iter=100	323	0.81
k-modes	nominal	k=7	45	0.77
HC-AGG	nominal	k=5,ward.D2	323	0.77
k-modes	nominal	k=7	323	0.76

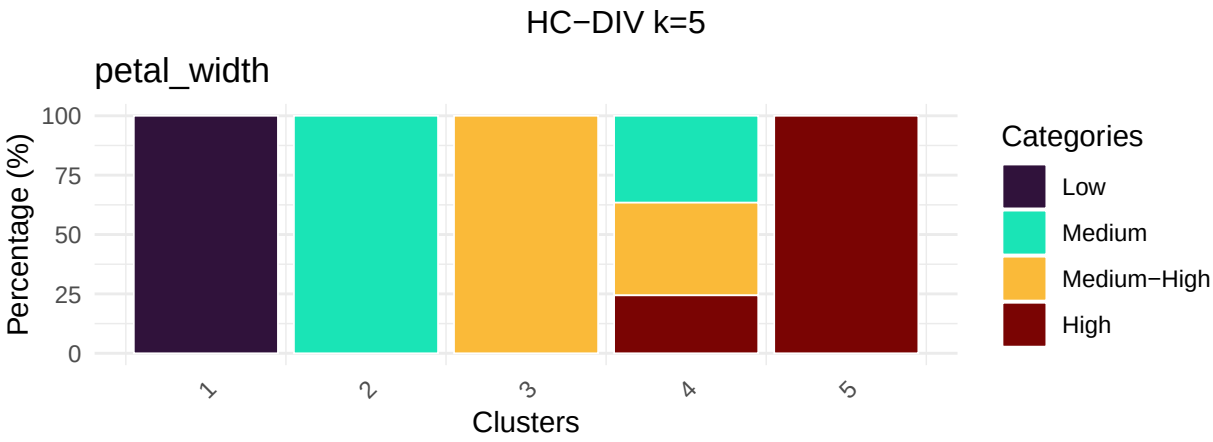
method_name	variables	parameters	rand.seed	homo_score
LCM	nominal	k=3,Binary_p_Ekjh	323	0.74
FMM	ordinal	k=3,EEI	323	0.73
k-modes	ordinal	k=3	45	0.62
k-modes	ordinal	k=3	323	0.27

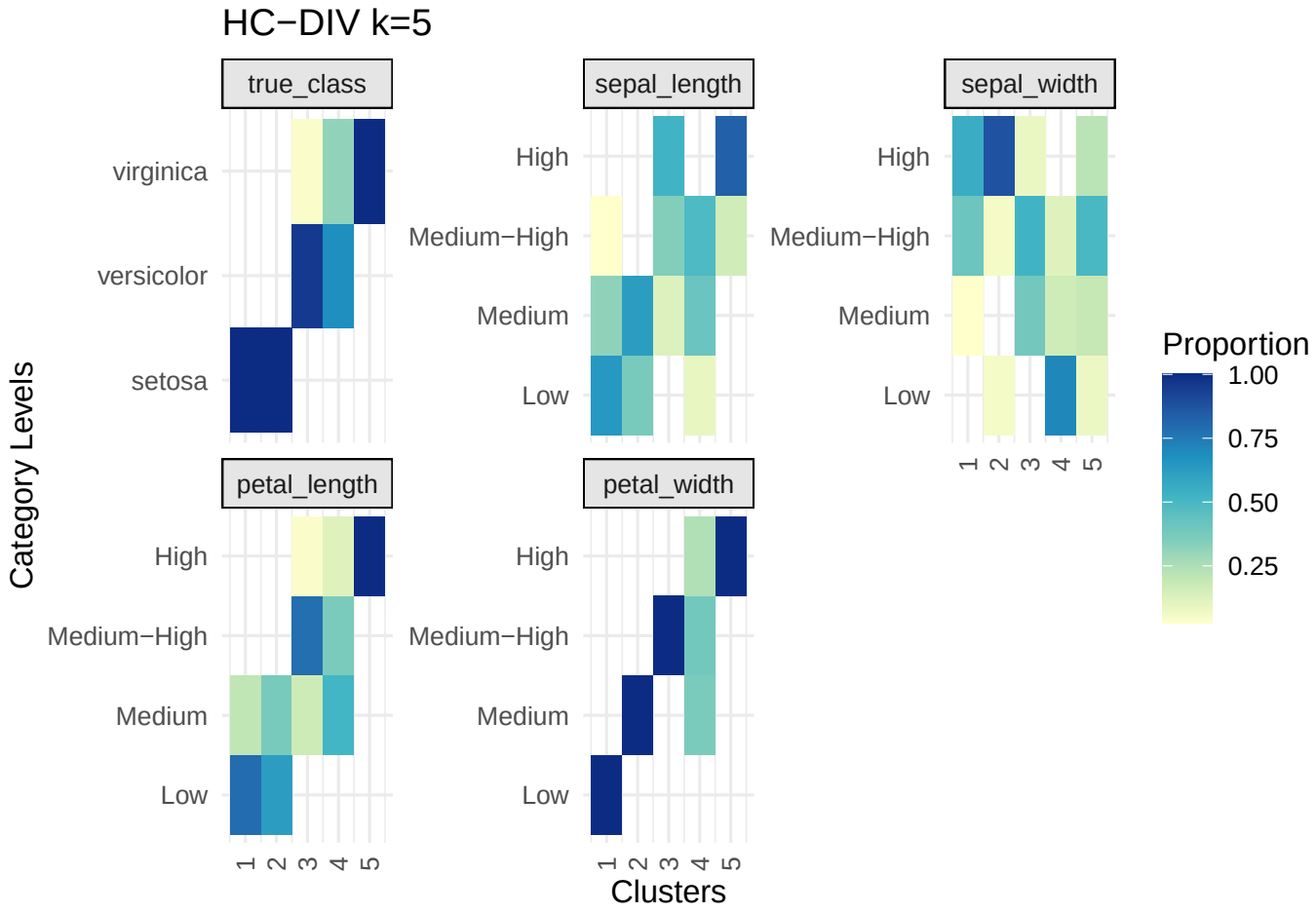
Iris: Cluster charts

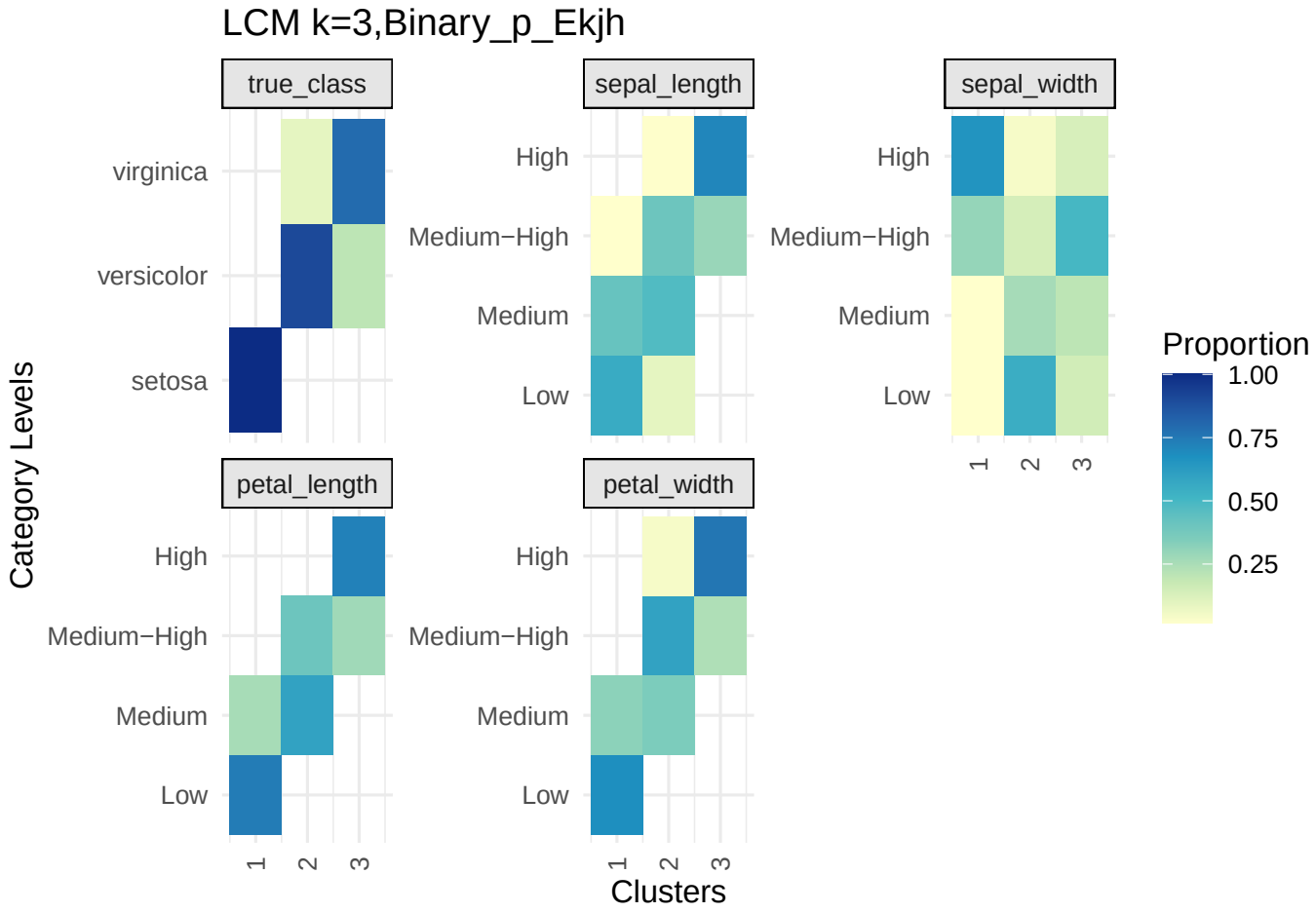
Although LCM chooses the correct number of clusters, the HC-DIV model results in the greatest or homogeneity, or cluster purity.




HC−DIV k=5
sepal_width
Percentage (%)
100
75
50
25
0
1
2
3
4
5
Clusters
Categories
Low
Medium
Medium−High
High
petal_length
Percentage (%)
100
75
50
25
0
1
2
3
4
5
Clusters
Categories
Low
Medium
Medium−High
High


HC−DIV k=5
petal_width
Percentage (%)
100
75
50
25
0
1
2
3
4
5
Clusters
Categories
Low
Medium
Medium−High
High


HC−DIV k=5
true_class
sepal_length
sepal_width
petal_length
petal_width
virginica
versicolor
setosa
High
Medium−High
Medium
Low
1
2
3
4
5
Category Levels
Clusters
Proportion
1.00
0.75
0.50
0.25


LCM k=3,Binary_p_Ekjh
true_class
sepal_length
sepal_width
petal_length
petal_width
virginica
versicolor
setosa
High
Medium−High
Medium
Low
1
2
3
Category Levels
Clusters
Proportion
1.00
0.75
0.50
0.25

Mushroom dataset

The mushroom dataset contains physical characteristics of mushrooms (8,124 instances, 22 attributes) classified as edible or poisonous. Every single feature is categorical (e.g., cap shape, gill color), making it great for our needs.

Table 14: Structure of Mushroom Dataset

Structure
'data.frame': 8124 obs. of 23 variables:
$ class : Factor w/ 2 levels "e","p": 2 1 1 2 1 1 1 1 2 1 ...
$ cap_shape : Factor w/ 6 levels "b","c","f","k",..: 6 6 1 6 6 6 1 1 6 1 ...
$ cap_surface : Factor w/ 4 levels "f","g","s","y": 3 3 3 4 3 4 3 4 4 3 ...
$ cap_color : Factor w/ 10 levels "b","c","e","g",..: 5 10 9 9 4 10 9 9 9 10 ...
$ bruises : Factor w/ 2 levels "f","t": 2 2 2 2 1 2 2 2 2 2 ...
$ odor : Factor w/ 9 levels "a","c","f","l",..: 7 1 4 7 6 1 1 4 7 1 ...
$ gill_attachment : Factor w/ 2 levels "a","f": 2 2 2 2 2 2 2 2 2 2 ...
$ gill_spacing : Factor w/ 2 levels "c","w": 1 1 1 1 2 1 1 1 1 1 ...
$ gill_size : Factor w/ 2 levels "b","n": 2 1 1 2 1 1 1 1 2 1 ...
$ gill_color : Factor w/ 12 levels "b","e","g","h",..: 5 5 6 6 5 6 3 6 8 3 ...
$ stalk_shape : Factor w/ 2 levels "e","t": 1 1 1 1 2 1 1 1 1 1 ...
$ stalk_root : Factor w/ 5 levels "?","b","c","e",..: 4 3 3 4 4 3 3 3 4 3 ...
$ stalk_surface_above_ring: Factor w/ 4 levels "f","k","s","y": 3 3 3 3 3 3 3 3 3 3 ...
$ stalk_surface_below_ring: Factor w/ 4 levels "f","k","s","y": 3 3 3 3 3 3 3 3 3 3 ...
$ stalk_color_above_ring : Factor w/ 9 levels "b","c","e","g",..: 8 8 8 8 8 8 8 8 8 8 ...
$ stalk_color_below_ring : Factor w/ 9 levels "b","c","e","g",..: 8 8 8 8 8 8 8 8 8 8 ...
$ veil_type : Factor w/ 1 level "p": 1 1 1 1 1 1 1 1 1 1 ...
$ veil_color : Factor w/ 4 levels "n","o","w","y": 3 3 3 3 3 3 3 3 3 3 ...
$ ring_number : Factor w/ 3 levels "n","o","t": 2 2 2 2 2 2 2 2 2 2 ...
$ ring_type : Factor w/ 5 levels "e","f","l","n",..: 5 5 5 5 1 5 5 5 5 5 ...
$ spore_print_color : Factor w/ 9 levels "b","h","k","n",..: 3 4 4 3 4 3 3 4 3 3 ...
$ population : Factor w/ 6 levels "a","c","n","s",..: 4 3 3 4 1 3 3 4 5 4 ...
$ habitat : Factor w/ 7 levels "d","g","l","m",..: 6 2 4 6 2 2 4 4 2 4 ...

Table 15: Exhaustive Summary Profile of All Categorical Features

Mushroom Feature	Factor Level Value	Count (N)	Distribution (%)
bruises	f (no)	4748	58.4%
bruises	t (bruises)	3376	41.6%
cap_color	n (brown)	2284	28.1%
cap_color	g (gray)	1840	22.6%
cap_color	e (red)	1500	18.5%
cap_color	y (yellow)	1072	13.2%
cap_color	w (white)	1040	12.8%
cap_color	b (buff)	168	2.1%
cap_color	p (pink)	144	1.8%
cap_color	c (cinnamon)	44	0.5%
cap_color	r (green)	16	0.2%
cap_color	u (purple)	16	0.2%
cap_shape	x (convex)	3656	45.0%
cap_shape	f (flat)	3152	38.8%

cap_shape	k (knobbed)	828	10.2%
cap_shape	b (bell)	452	5.6%
cap_shape	s (sunken)	32	0.4%
cap_shape	c (conical)	4	0.0%
cap_surface	y (scaly)	3244	39.9%
cap_surface	s (smooth)	2556	31.5%
cap_surface	f (fibrous)	2320	28.6%
cap_surface	g (grooves)	4	0.0%
gill_attachment	f (free)	7914	97.4%
gill_attachment	a (attached)	210	2.6%
gill_color	b (buff)	1728	21.3%
gill_color	p (pink)	1492	18.4%
gill_color	w (white)	1202	14.8%
gill_color	n (brown)	1048	12.9%
gill_color	g (gray)	752	9.3%
gill_color	h (chocolate)	732	9.0%
gill_color	u (purple)	492	6.1%
gill_color	k (black)	408	5.0%
gill_color	e (red)	96	1.2%
gill_color	y (yellow)	86	1.1%
gill_color	o (orange)	64	0.8%
gill_color	r (green)	24	0.3%
gill_size	b (broad)	5612	69.1%
gill_size	n (narrow)	2512	30.9%
gill_spacing	c (close)	6812	83.9%
gill_spacing	w (crowded)	1312	16.1%
habitat	d (woods)	3148	38.7%
habitat	g (grasses)	2148	26.4%
habitat	p (paths)	1144	14.1%
habitat	l (leaves)	832	10.2%
habitat	u (urban)	368	4.5%
habitat	m (meadows)	292	3.6%
habitat	w (waste)	192	2.4%
odor	n (none)	3528	43.4%
odor	f (foul)	2160	26.6%
odor	s (spicy)	576	7.1%
odor	y (fishy)	576	7.1%
odor	a (almond)	400	4.9%
odor	l (anise)	400	4.9%
odor	p (pungent)	256	3.2%
odor	c (creosote)	192	2.4%
odor	m (musty)	36	0.4%
population	v (several)	4040	49.7%
population	y (solitary)	1712	21.1%
population	s (scattered)	1248	15.4%
population	n (numerous)	400	4.9%
population	a (abundant)	384	4.7%
population	c (clustered)	340	4.2%
ring_number	o (one)	7488	92.2%
ring_number	t (two)	600	7.4%

ring_number	n (none)	36	0.4%
ring_type	p (pendant)	3968	48.8%
ring_type	e (evanescent)	2776	34.2%
ring_type	l (large)	1296	16.0%
ring_type	f (flaring)	48	0.6%
ring_type	n (none)	36	0.4%
spore_print_color	w (white)	2388	29.4%
spore_print_color	n (brown)	1968	24.2%
spore_print_color	k (black)	1872	23.0%
spore_print_color	h (chocolate)	1632	20.1%
spore_print_color	r (green)	72	0.9%
spore_print_color	b (buff)	48	0.6%
spore_print_color	y (yellow)	48	0.6%
spore_print_color	u (purple)	48	0.6%
spore_print_color	o (orange)	48	0.6%
stalk_color_above_ring	w (white)	4464	54.9%
stalk_color_above_ring	p (pink)	1872	23.0%
stalk_color_above_ring	g (gray)	576	7.1%
stalk_color_above_ring	n (brown)	448	5.5%
stalk_color_above_ring	b (buff)	432	5.3%
stalk_color_above_ring	o (orange)	192	2.4%
stalk_color_above_ring	e (red)	96	1.2%
stalk_color_above_ring	c (cinnamon)	36	0.4%
stalk_color_above_ring	y (yellow)	8	0.1%
stalk_color_below_ring	w (white)	4384	54.0%
stalk_color_below_ring	p (pink)	1872	23.0%
stalk_color_below_ring	g (gray)	576	7.1%
stalk_color_below_ring	n (brown)	512	6.3%
stalk_color_below_ring	b (buff)	432	5.3%
stalk_color_below_ring	o (orange)	192	2.4%
stalk_color_below_ring	e (red)	96	1.2%
stalk_color_below_ring	c (cinnamon)	36	0.4%
stalk_color_below_ring	y (yellow)	24	0.3%
stalk_root	b (bulbous)	3776	46.5%
stalk_root	? (missing)	2480	30.5%
stalk_root	e (equal)	1120	13.8%
stalk_root	c (club)	556	6.8%
stalk_root	r (rooted)	192	2.4%
stalk_shape	t (tapering)	4608	56.7%
stalk_shape	e (enlarging)	3516	43.3%
stalk_surface_above_ring	s (smooth)	5176	63.7%
stalk_surface_above_ring	k (silky)	2372	29.2%
stalk_surface_above_ring	f (fibrous)	552	6.8%
stalk_surface_above_ring	y (scaly)	24	0.3%
stalk_surface_below_ring	s (smooth)	4936	60.8%
stalk_surface_below_ring	k (silky)	2304	28.4%
stalk_surface_below_ring	f (fibrous)	600	7.4%
stalk_surface_below_ring	y (scaly)	284	3.5%
veil_color	w (white)	7924	97.5%
veil_color	n (brown)	96	1.2%

veil_color	o (orange)	96	1.2%
veil_color	y (yellow)	8	0.1%
veil_type	p (partial)	8124	100.0%

Mushroom: Feature engineering

Unlike the Iris data, Mushrooms data is relatively more messy. This can adversely affect our clustering results by adding noise. We must clean the data before clustering.

Table 16: Mushroom data

variable_name	num_missing
cap_shape	0
cap_surface	0
cap_color	0
bruises	0
odor	0
gill_attachment	0
gill_spacing	0
gill_size	0
gill_color	0
stalk_shape	0
stalk_root	0
stalk_surface_above_ring	0
stalk_surface_below_ring	0
stalk_color_above_ring	0
stalk_color_below_ring	0
veil_type	0
veil_color	0
ring_number	0
ring_type	0
spore_print_color	0
population	0
habitat	0

There is no missing data.

We may consider removing or regrouping variables with "low" or "high" cell counts. Certainly, variables with constant values are of no value as they only complicate the process of find good results.

Table 17: Mushroom data, more than 90% in a cell

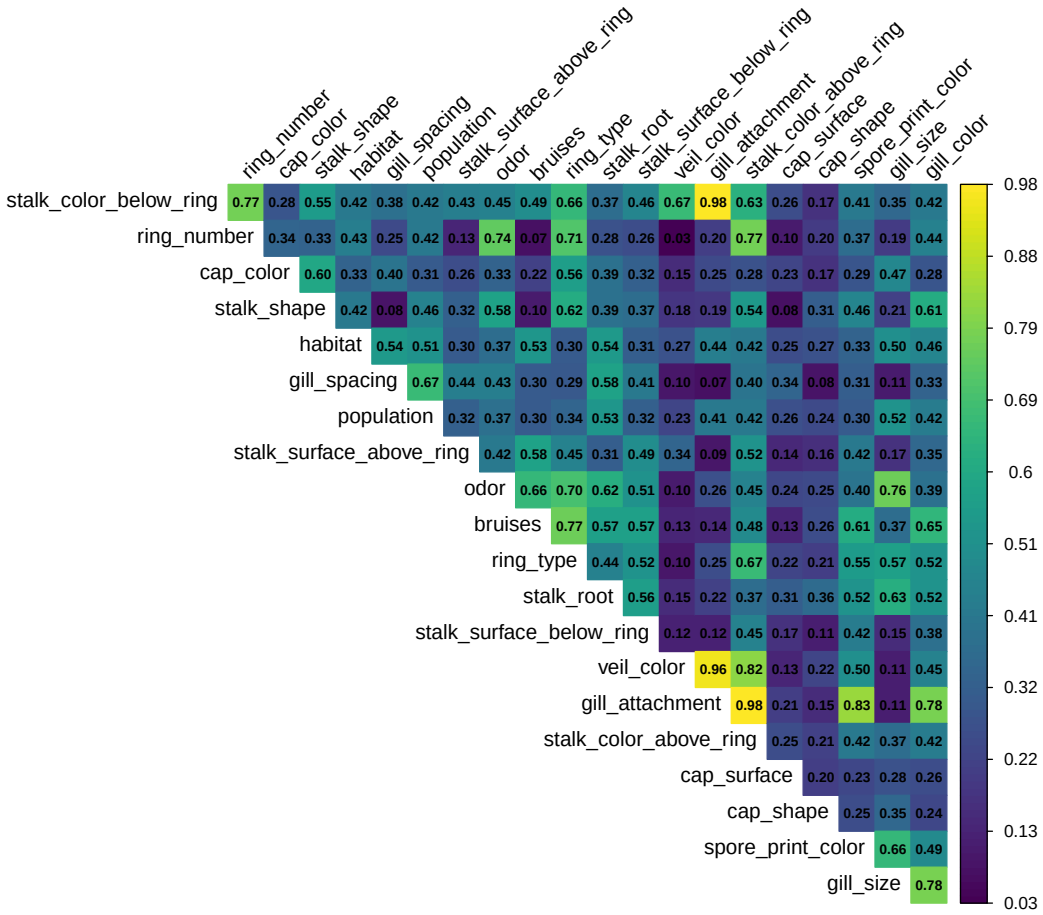
Variable	Level_Expanded	Count	Percentage	percentage
gill_attachment	f (free)	7914	97.4%	0.974
ring_number	o (one)	7488	92.2%	0.922
veil_color	w (white)	7924	97.5%	0.975
veil_type	p (partial)	8124	100.0%	1.000

Remove **`veil_type`** since it doesn't have any variation and, therefore, doesn't add any useful information for determining clusters. Let's investigate the other variables that were flagged.

Variable	Level_Expanded	Count	Percentage
gill_attachment	f (free)	7914	97.4%
gill_attachment	a (attached)	210	2.6%

Variable	Level_Expanded	Count	Percentage
veil_color	w (white)	7924	97.5%
veil_color	n (brown)	96	1.2%
veil_color	o (orange)	96	1.2%
veil_color	y (yellow)	8	0.1%

Variable	Level_Expanded	Count	Percentage
ring_number	o (one)	7488	92.2%
ring_number	t (two)	600	7.4%
ring_number	n (none)	36	0.4%



While the original Audubon Field Guide provides an unweighted checklist of traits, subsequent algorithmic analysis demonstrates that the feature space is heavily anchored by a single variable: `odor` [Lantz, 2013]. Rule-induction and decision tree models consistently leverage `odor` as the root partition due to its high mutual information content with the underlying biological structures [Lantz, 2013].

When analyzing this data in an unsupervised context (with the `class` column removed), the 9 unique odor labels compress into 4 primary behavior clusters based on how they naturally align with secondary physical traits:

1. **The Chemical Warnings:** Foul, fishy, spicy, and pungent mushrooms. These present high structural similarity and map almost exclusively to poisonous species.
2. **The Sweet Aromas:** Almond and anise mushrooms. These hold highly specific cap/ring structures and map uniformly to edible profiles.
3. **The Neutrals:** Odorless mushrooms. This is the largest group in the dataset. Because they lack a distinct smell, they form a massive, mixed profile that must rely on secondary features (like spore print

color) to segment further.
4. **The Outliers:** Creosote and musty mushrooms. These capture tiny, niche biological variations that do not cleanly align with the other three major groups.

Therefore, initializing an unsupervised algorithm (such as K-Modes) with `modes = 4` serves logical baseline for initial feature investigation.

Table 18: Mushroom feature importance

feature	ChiSq_Score	max_corr	max_corr_with
ring_type	12928.05	0.77	bruises
spore_print_color	11486.29	0.83	gill_attachment
gill_color	10706.42	0.78	gill_size
stalk_root	9768.81	0.63	gill_size
odor	9544.19	0.76	gill_size
stalk_color_above_ring	7690.27	0.98	gill_attachment
stalk_color_below_ring	7433.00	0.98	gill_attachment
habitat	6962.98	0.54	gill_spacing
population	6896.74	0.67	gill_spacing
stalk_surface_above_ring	6212.21	0.58	bruises
cap_color	6059.49	0.60	stalk_shape
bruises	5990.21	0.77	ring_type
stalk_surface_below_ring	5882.58	0.57	bruises
gill_spacing	4930.40	0.67	population
gill_size	4675.13	0.78	gill_color
stalk_shape	2531.35	0.62	ring_type
cap_surface	2433.81	0.34	gill_spacing
cap_shape	1776.33	0.36	stalk_root
ring_number	711.22	0.77	stalk_color_below_ring
veil_color	207.25	0.96	gill_attachment
gill_attachment	141.06	0.98	stalk_color_above_ring
gill_attachment	141.06	0.98	stalk_color_below_ring

The feature importance and Cramer V correlations confirms our initial suspicions based on the percentages of the variable values. We drop **`gill_attachment`**, **`veil_color`**, **`ring_number`**. Additionally, we remove **`gill_size`**, **`bruises`**.

Table 19: Mushroom feature importance

feature	ChiSq_Score	max_corr	max_corr_with
ring_type	12928.05	0.70	odor
spore_print_color	11486.29	0.55	ring_type
gill_color	10706.42	0.61	stalk_shape
stalk_root	9768.81	0.62	odor
odor	9544.19	0.70	ring_type
stalk_color_above_ring	7690.27	0.67	ring_type
stalk_color_below_ring	7433.00	0.66	ring_type
habitat	6962.98	0.54	gill_spacing
population	6896.74	0.67	gill_spacing
stalk_surface_above_ring	6212.21	0.52	stalk_color_above_ring
cap_color	6059.49	0.60	stalk_shape
stalk_surface_below_ring	5882.58	0.56	stalk_root

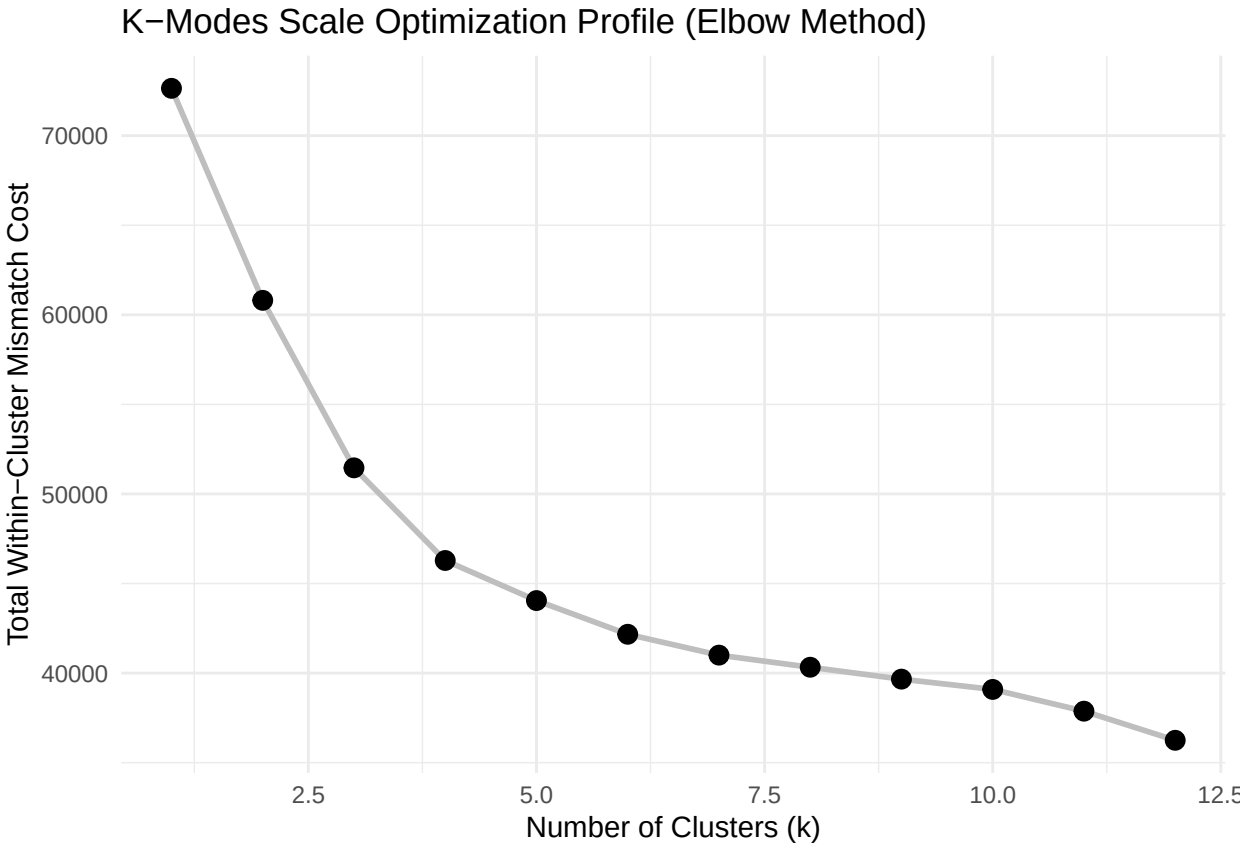
feature	ChiSq_Score	max_corr	max_corr_with
gill_spacing	4930.40	0.67	population
stalk_shape	2531.35	0.62	ring_type
cap_surface	2433.81	0.34	gill_spacing
cap_shape	1776.33	0.36	stalk_root

Let's turn to model fitting.

Mushroom: k-modes

The Mushroom dataset is many times larger than the recoded Iris dataset we started with, with dimension 8124, 17. Increasing large data dimensions presents challenges against using k-modes which I don't think is necessary to explore, at least not in this paper. We use Huang's fast frequency initiation.

```
# Isolate target keys and set search grid bounds
possible_k <- 1:12
mismatch_costs <- numeric(length(possible_k))

# Vectorized loop running on fast configuration flags
for (i in seq_along(possible_k)) {
  set.seed(42)
  fit <-  klaR::kmodes(
    data = mushroom_features,
    modes = possible_k[i],
    fast = TRUE, # Uses Huang's fast frequency initiation
    iter.max = 20
  )
  # Sum the total matching differences across all clusters
  mismatch_costs[i] <- sum(fit$withindiff)
}

# Create the evaluation dataframe
elbow_data <- data.frame(k = possible_k, Cost = mismatch_costs)

# Instantly render the Elbow graph
ggplot(elbow_data, aes(x = k, y = Cost)) +
  geom_line(linewidth = 1, color = "grey") +
  geom_point(size = 3, color = "black") +
  theme_minimal() +
  labs(
    title = "K-Modes Scale Optimization Profile (Elbow Method)",
    x = "Number of Clusters (k)",
    y = "Total Within-Cluster Mismatch Cost"
  )
```



The elbow plot suggests 6 clusters and so we run a bagged k-modes.

Table 20: Mushrooms data: Bagged k=modes predicted clusters

	1	2	3	4	5	6
e	1516	71	325	0	270	2026
p	501	1756	20	1320	45	274

Table 21: Mushroom: Model performance

method_name	variables	parameters	rand.seed	homo_score
bagged k-modes	nominal	k=7,boot=100	42	0.56

Mushroom: HC

For hierarchical clustering, we first get the most representative modes then we run Gower's distance on these modes to use in the clustering algorithm. Effectively we are doing the following:

1. Get "representative" modes.
2. Perform HC on these modes, assigning each to one of k clusters.
3. Map original data to one of the "representative" modes.
4. Therefore, the original data rows are mapped to one of k clusters.

Let's start by finding the number of optimal clusters k using 100 modes to keep the execution fast.

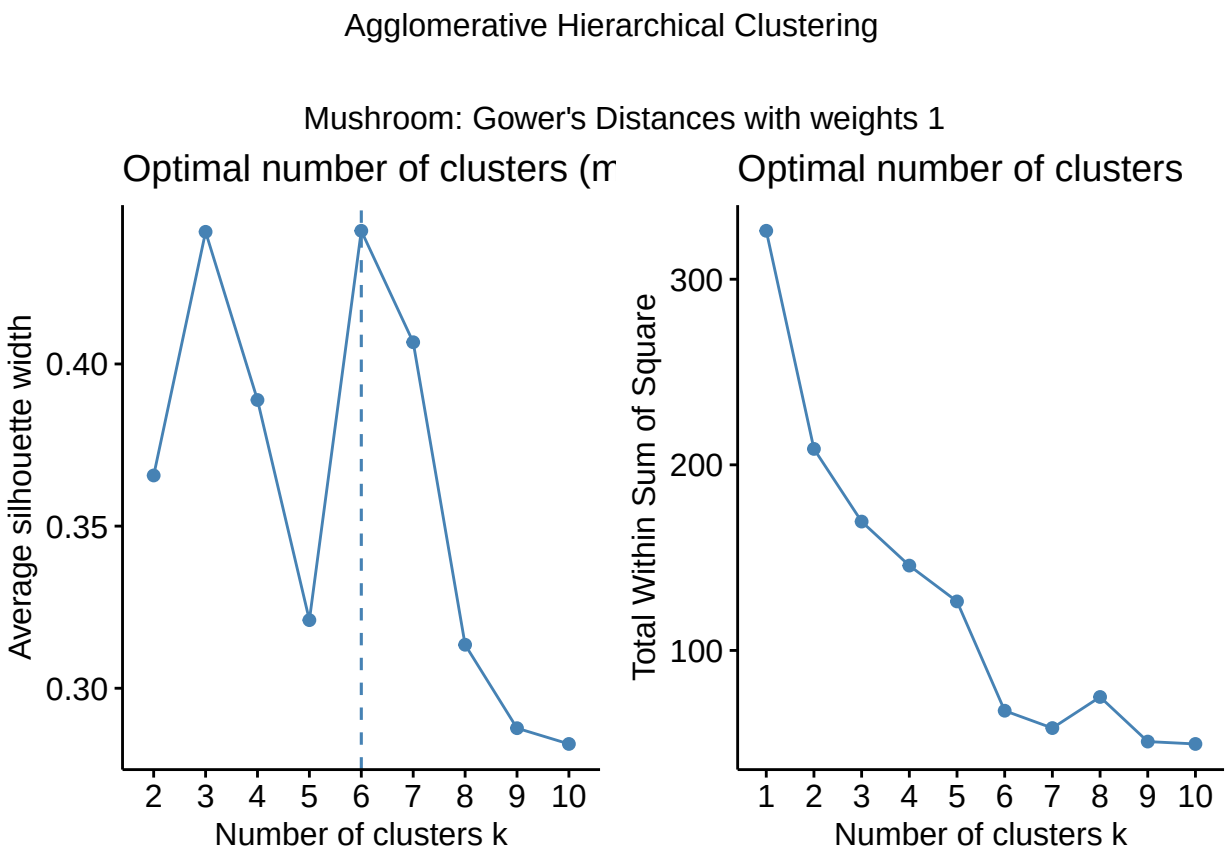


Table 22: HC-AGG,k=5,FAST

	1	2	3	4	5
e	1905	768	0	1534	1
p	400	0	1753	449	1314

Table 23: HC-AGG,k=8,FAST

	1	2	3	4	5	6	7	8
e	1904	1	768	0	805	1	441	288
p	112	288	0	1753	411	1314	37	1

Table 24: Mushroom: Model performance

method_name	variables	parameters	rand.seed	homo_score
bagged k-modes	nominal	k=7,boot=100	42	0.56
HC-AGG	nominal	k=5,FAST,100 Modes	42	0.62
HC-AGG	nominal	k=8,FAST,100 Modes	42	0.76

Now, divisive hierarchical clustering.

Table 25: HC-DIV,k=5,FAST

	1	2	3	4	5
e	2710	768	56	1	673
p	811	0	1754	1314	37

Table 26: HC-DIV,k=8,FAST

	1	2	3	4	5	6	7	8
e	1875	75	768	56	760	1	385	288
p	190	288	0	1754	333	1314	36	1

Table 27: Mushroom: Model performance

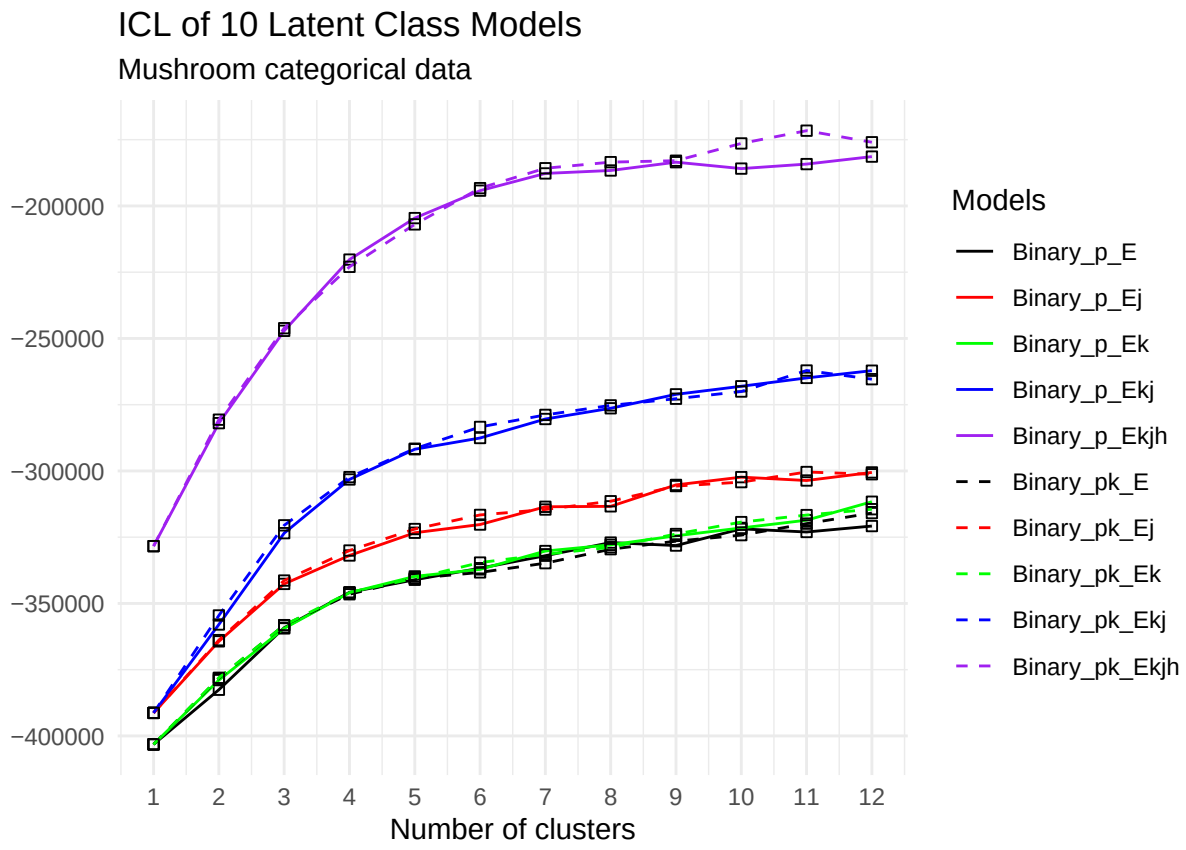
method_name	variables	parameters	rand.seed	homo_score
bagged k-modes	nominal	k=7,boot=100	42	0.56
HC-AGG	nominal	k=5,FAST,100 Modes	42	0.62
HC-AGG	nominal	k=8,FAST,100 Modes	42	0.76
HC-DIV	nominal	k=5,FAST,100 Modes	42	0.59
HC-DIV	nominal	k=8,FAST,100 Modes	42	0.67

Mushroom: LCM

```
mushrooms_lcm_dir <- file.path(intermediates_dir,"mushrooms","lcm")
if (run_shroom_lcm_models){
names_possible_models <- c("Binary_p_E","Binary_p_Ej", "Binary_p_Ek",
                           "Binary_p_Ekj","Binary_p_Ekjh","Binary_pk_E",
                           "Binary_pk_Ej","Binary_pk_Ek","Binary_pk_Ekj",
                           "Binary_pk_Ekjh")

names(names_possible_models) <- names_possible_models
set.seed(42)
mushroom_lcm_models <-
  purrr::map(names_possible_models,function(x){
    res <- Rmixmod::mixmodCluster(
      mushroom_features,
      nbCluster = 1:12,
      dataType = "qualitative",
      model = Rmixmod::mixmodMultinomialModel(listModels = x),
      criterion = c("ICL")
      )
    res
  })
dir.create(mushrooms_lcm_dir,recursive = TRUE)
  saveRDS(
    mushroom_lcm_models,
    file.path(mushrooms_lcm_dir,"mushroom_lcm_models.rds"))
} else {
```

```
  mushroom_lcm_models <- readRDS(
    file.path(mushrooms_lcm_dir,"mushroom_lcm_models.rds")
    )
}
```

Now we get the ICL values by cluster then plot.



The best model is `Binary_pk_Ejkh` at 11 clusters.

```
mushroom_lcm_models_best <- mushroom_lcm_models[["Binary_pk_Ekjh"]]@bestResult
```

Table 28: Mushroom: Predicted clusters LCM Binary_pk_Ekjh

	1	2	3	4	5	6	7	8	9	10	11
e	608	768	192	0	384	1728	288	96	0	0	144
p	256	0	0	1296	44	0	0	192	1728	288	112

Table 29: Mushroom: Methods performance

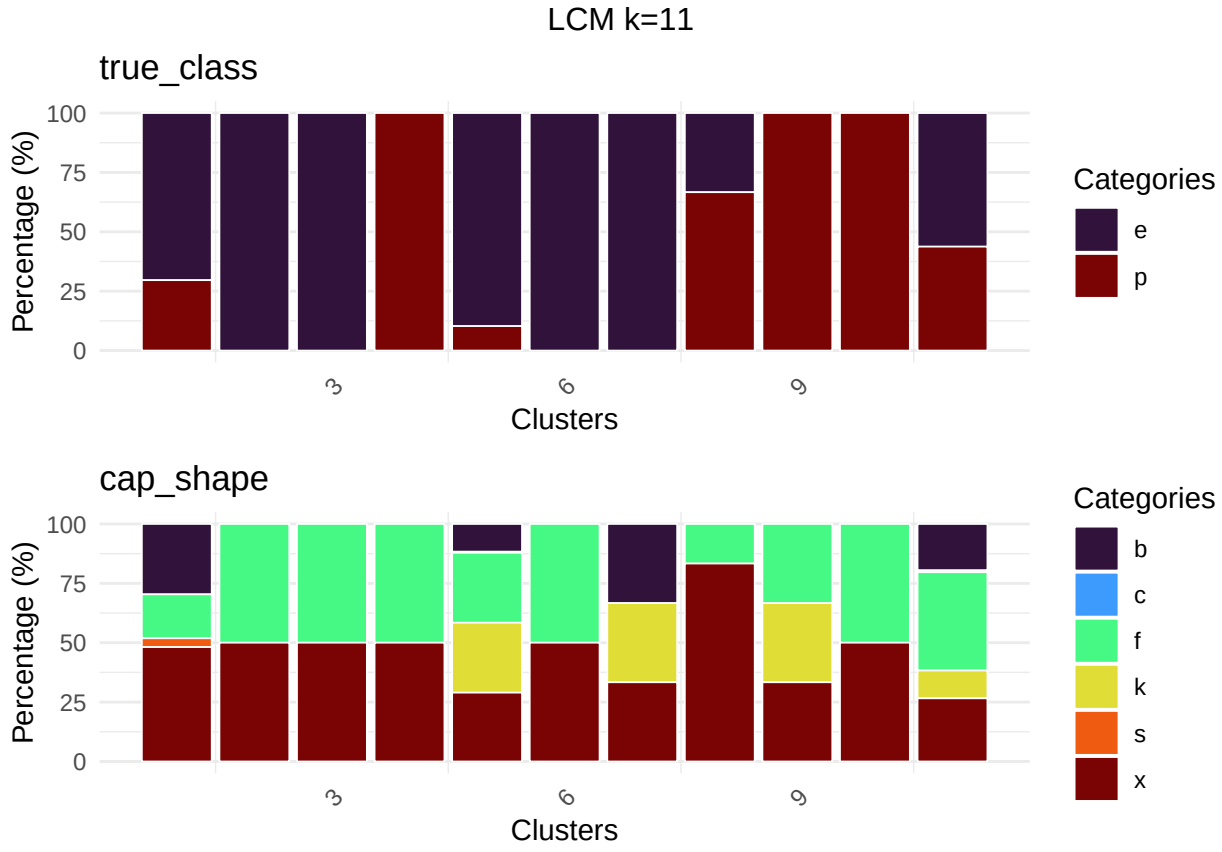
method_name	variables	parameters	rand.seed	homo_score
LCM	nominal	k=11,Binary_pk_Ekjh	42	0.82
HC-AGG	nominal	k=8,FAST,100 Modes	42	0.76

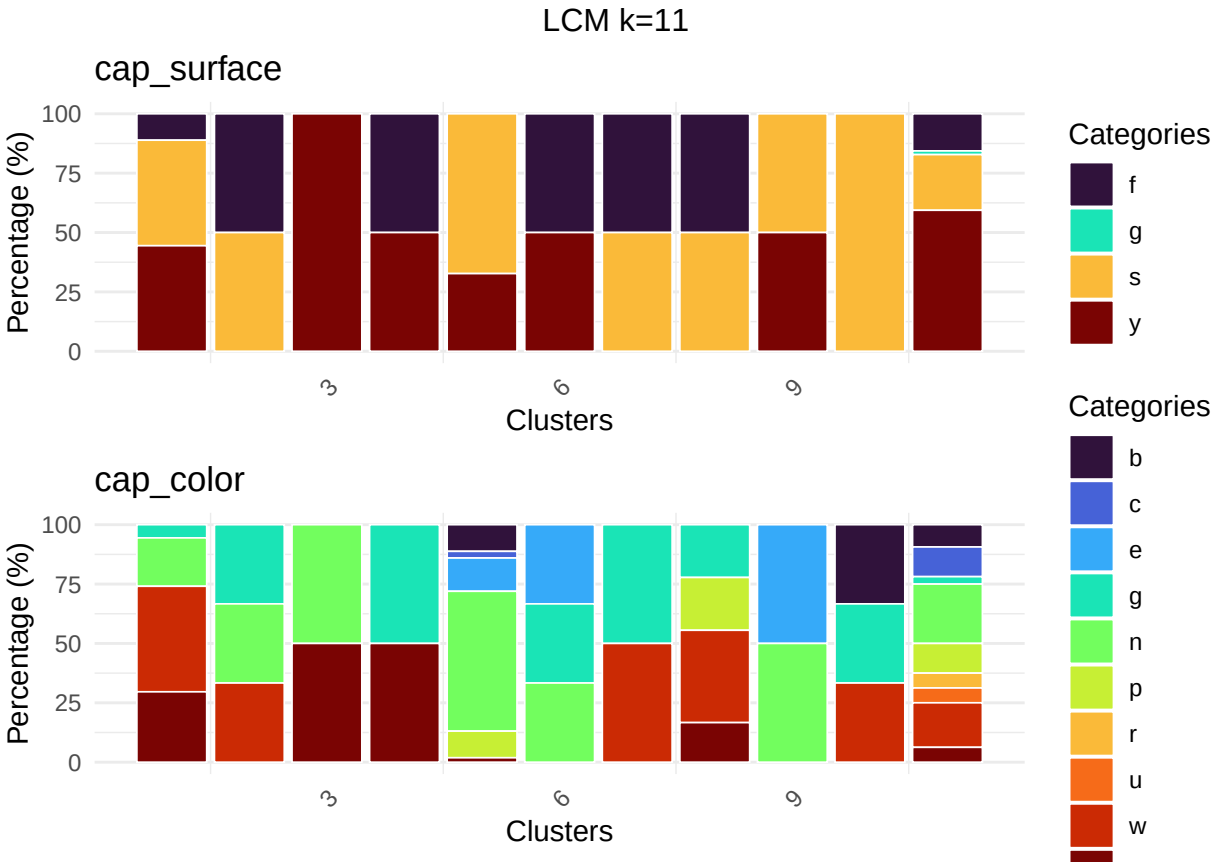
method_name	variables	parameters	rand.seed	homo_score
HC-DIV	nominal	k=8,FAST,100 Modes	42	0.67
HC-AGG	nominal	k=5,FAST,100 Modes	42	0.62
HC-DIV	nominal	k=5,FAST,100 Modes	42	0.59
bagged k-modes	nominal	k=7,boot=100	42	0.56

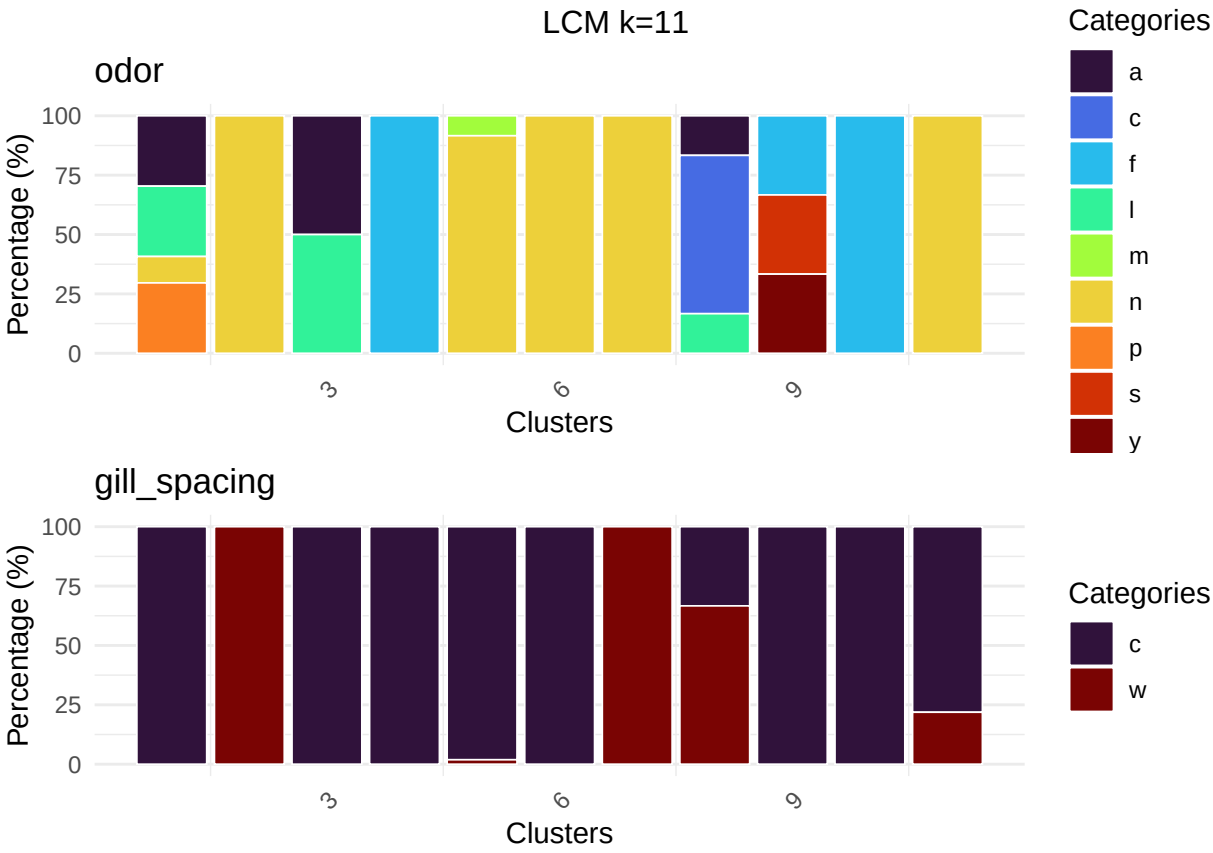
Of these models, LCM outperformed.

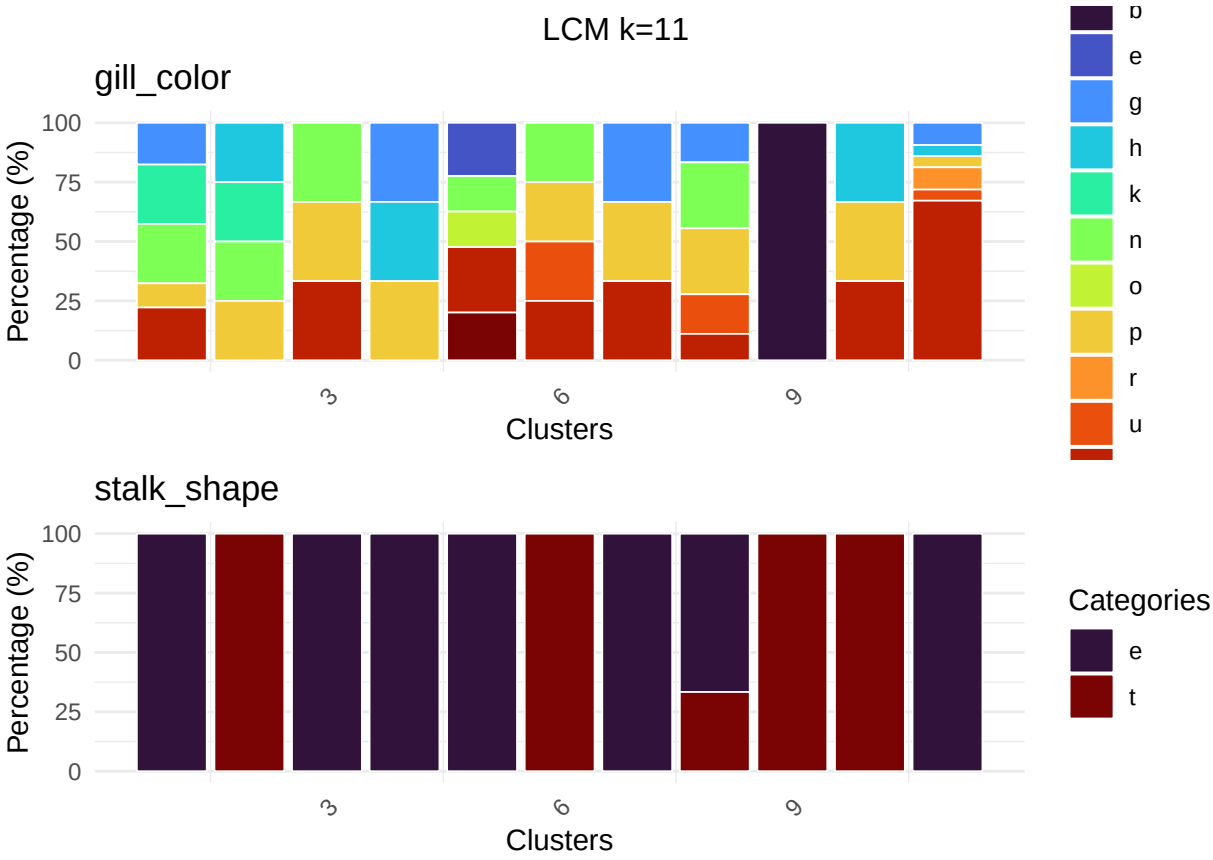
Mushroom: Cluster charts

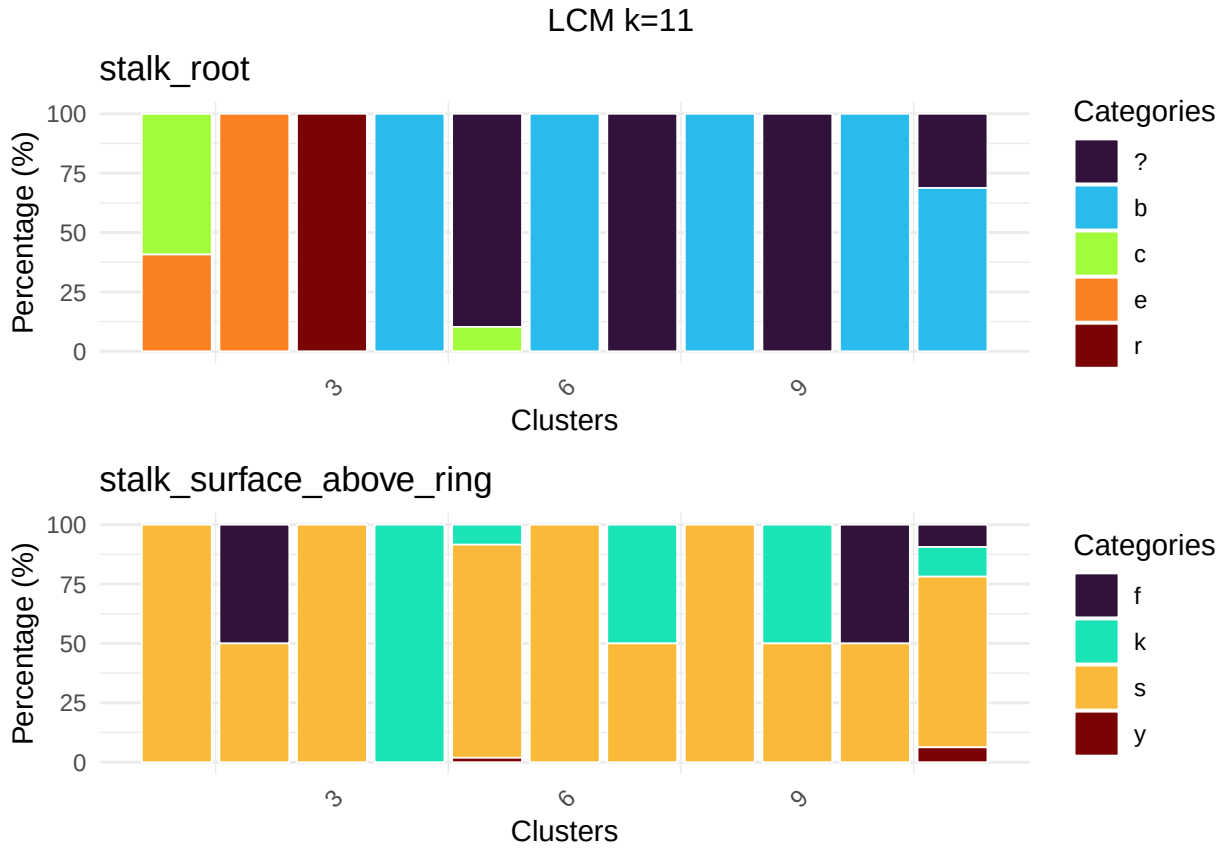
Clusters 4,9, and 10 have 100% purity with poisonous mushrooms and characterize the overwhelming majority of them with counts of 1296, 1728, 288, respectively. Thinking about cluster 9, the cluster with largest number of poisonous mushrooms always had "buff" colored gills, "evanescent" ring types, with odors being "foul", "spicy", or "fishy". In fact, these odors are only associated with poisonous mushrooms. Additionally, "large" rings are strictly associated with poisonous mushrooms, being the only ring type of cluster 4.

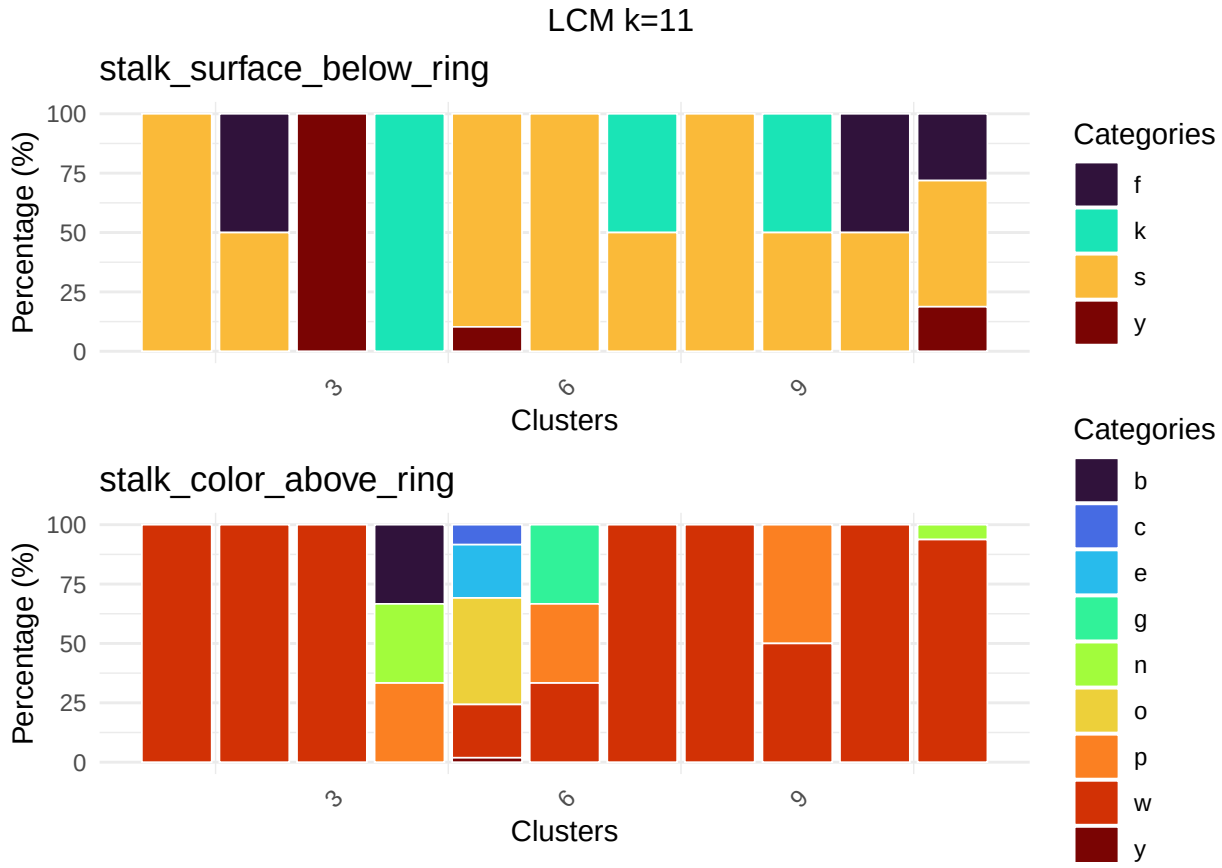


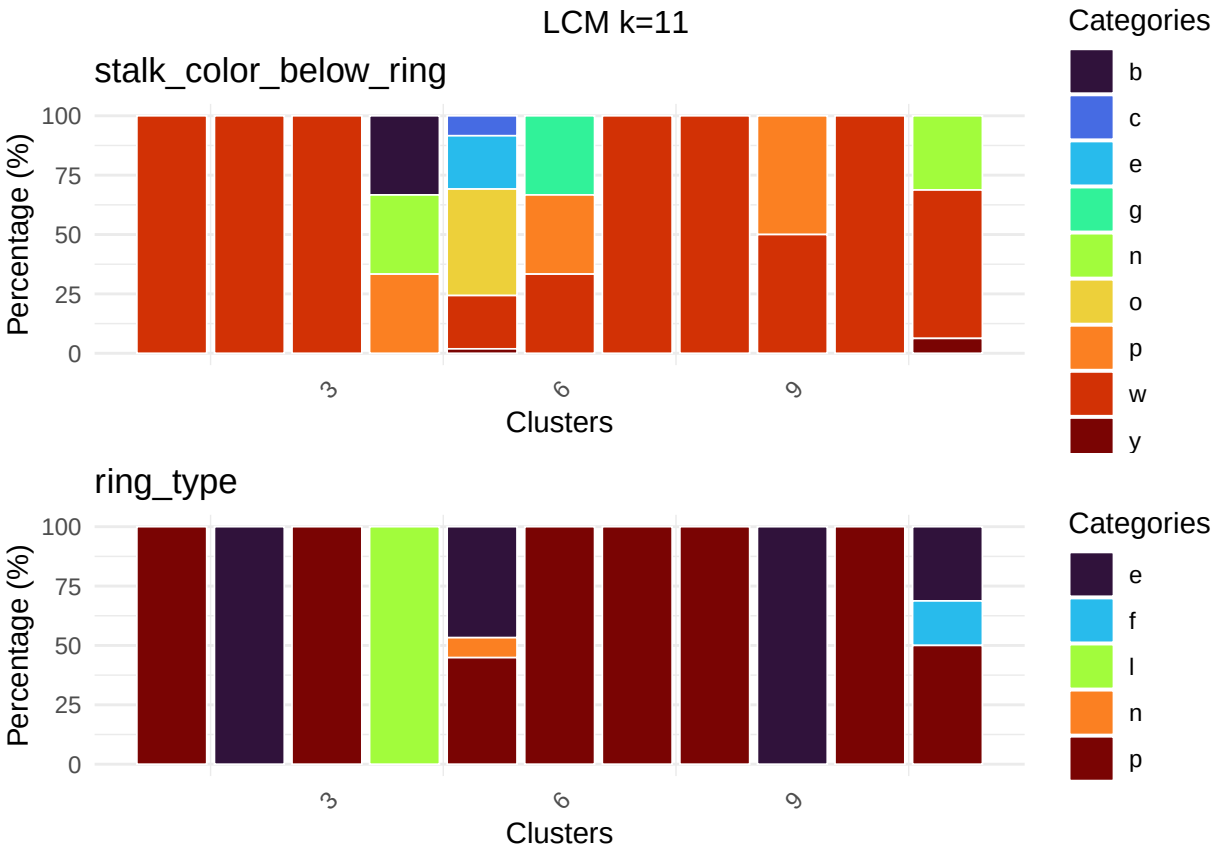

LCM k=11
cap_surface
Percentage (%)
100
75
50
25
0
3
6
9
Clusters
Categories
f
g
s
y
cap_color
Percentage (%)
100
75
50
25
0
3
6
9
Clusters
Categories
b
c
e
g
n
p
r
u
w

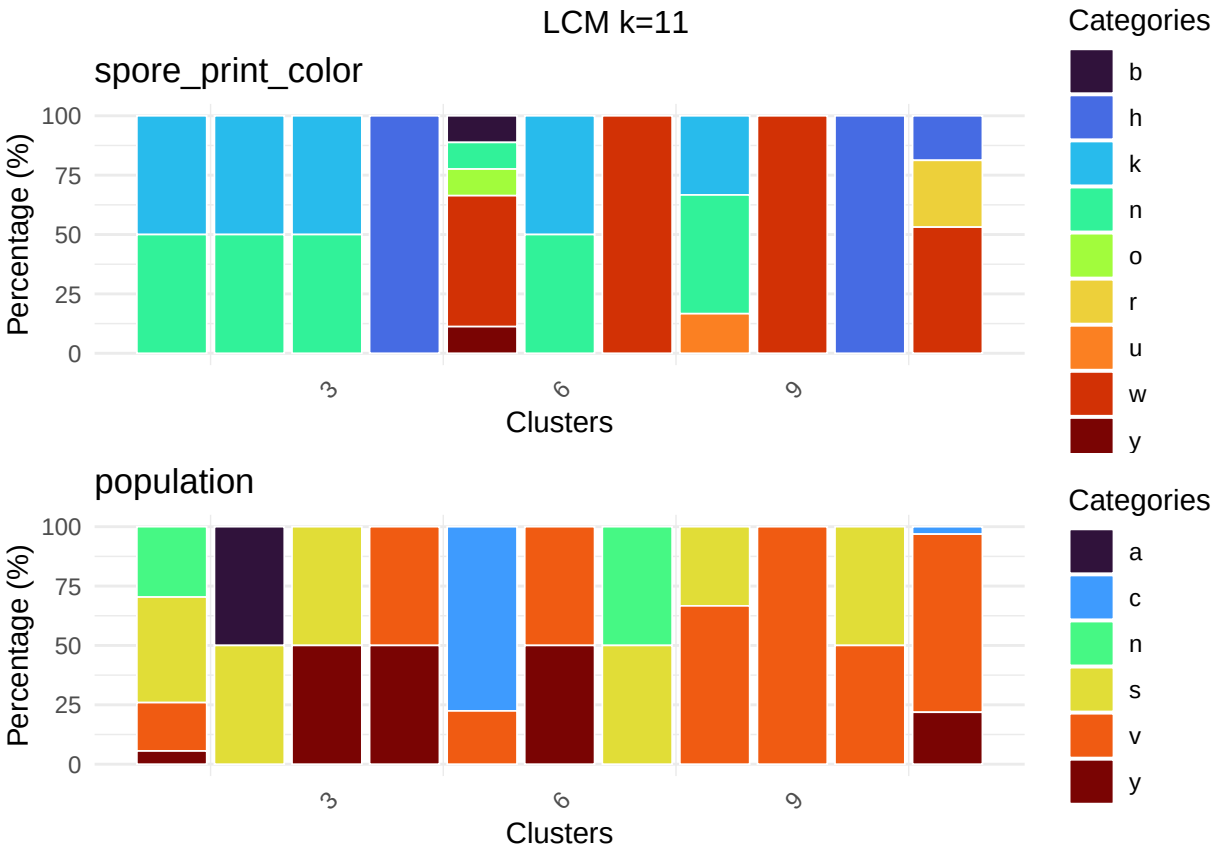

LCM k=11
odor
Percentage (%)
100
75
50
25
0
3
6
9
Clusters
Categories
a
c
f
l
m
n
p
s
y
gill_spacing
Percentage (%)
100
75
50
25
0
3
6
9
Clusters
Categories
c
w

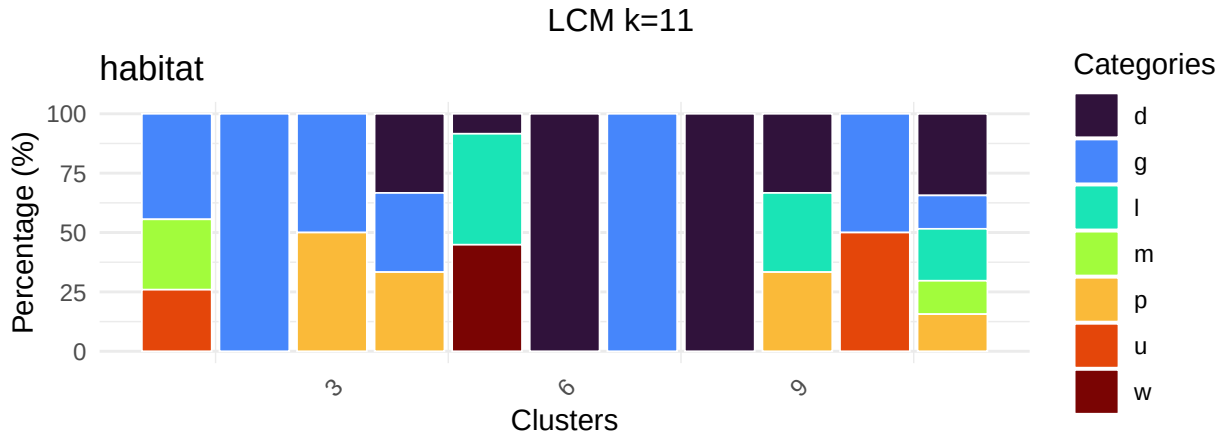

LCM k=11
gill_color
Percentage (%)
100
75
50
25
0
3
6
9
Clusters
b
e
g
h
k
n
o
p
r
u
stalk_shape
Percentage (%)
100
75
50
25
0
3
6
9
Clusters
Categories
e
t


LCM k=11
stalk_root
Percentage (%)
100
75
50
25
0
3
6
9
Clusters
Categories
?
b
c
e
r
stalk_surface_above_ring
Percentage (%)
100
75
50
25
0
3
6
9
Clusters
Categories
f
k
s
y


LCM k=11
stalk_surface_below_ring
Percentage (%)
100
75
50
25
0
3
6
9
Clusters
Categories
f
k
s
y
stalk_color_above_ring
Percentage (%)
100
75
50
25
0
3
6
9
Clusters
Categories
b
c
e
g
n
o
p
w
y


LCM k=11
stalk_color_below_ring
Percentage (%)
100
75
50
25
0
3
6
9
Clusters
Categories
b
c
e
g
n
o
p
w
y
ring_type
Percentage (%)
100
75
50
25
0
3
6
9
Clusters
Categories
e
f
l
n
p


LCM k=11
spore_print_color
Percentage (%)
100
75
50
25
0
3
6
9
Clusters
Categories
b
h
k
n
o
r
u
w
y
population
Percentage (%)
100
75
50
25
0
3
6
9
Clusters
Categories
a
c
n
s
v
y



Zoo dataset

The Zoo dataset is a classic benchmarking dataset containing 101 animal instances evaluated across 18 total columns [Forsyth, 1990]. It is heavily utilized for evaluating classification and unsupervised clustering tasks because its physical traits naturally group animals into distinct biological families.

```
zoo_data <- get(data("Zoo", package = "mlbench"))
zoo_features <- zoo_data %>%
  dplyr::select(-type) %>%
  dplyr::as_tibble() %>%
  dplyr::mutate(dplyr::across(everything(),as.factor)) %>%
  as.data.frame()
true_labels_zoo <- zoo_data$type
```

Zoo: k-modes

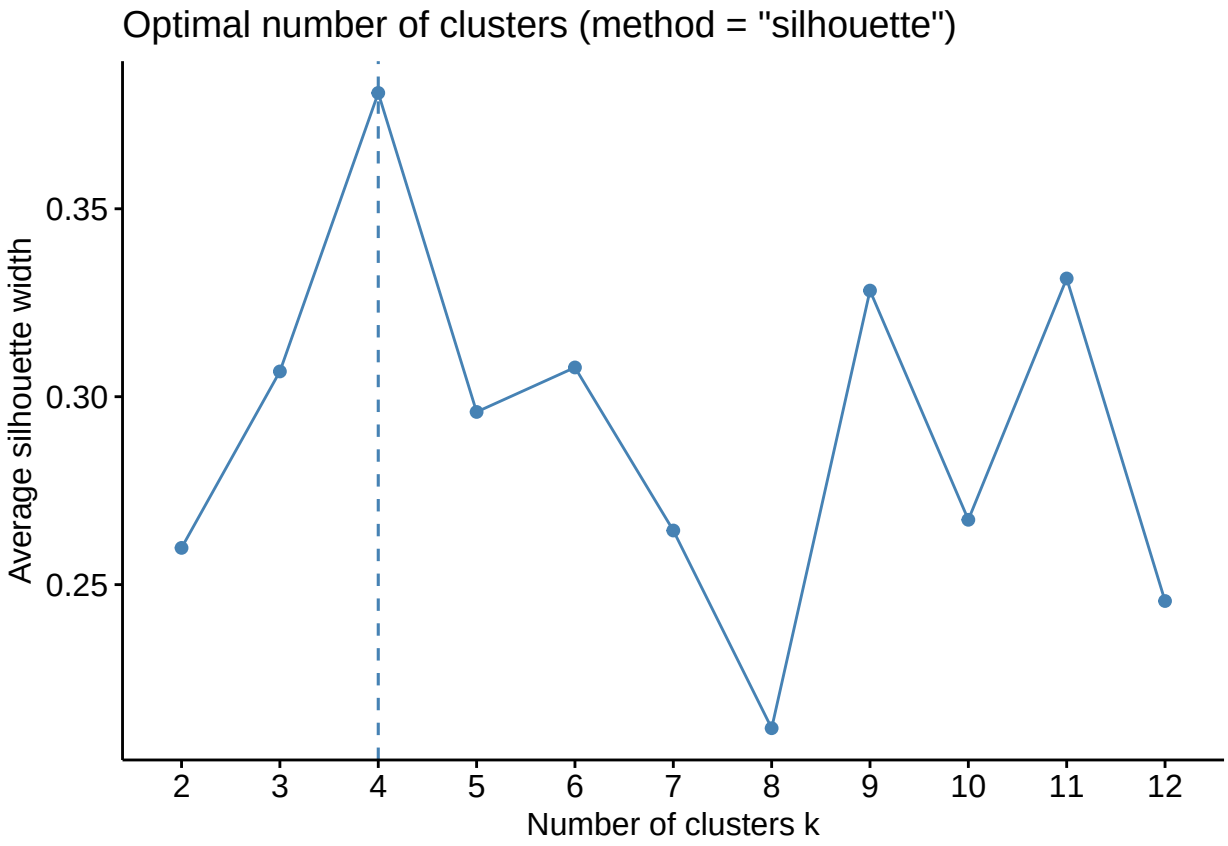


Table 30: Bagged k-modes 4 clusters: truth vs predicted

	1	2	3	4
mammal	0	2	0	39
bird	20	0	0	0
reptile	1	4	0	0
fish	0	13	0	0
amphibian	0	3	1	0
insect	0	0	8	0
mollusc.et.al	0	1	9	0

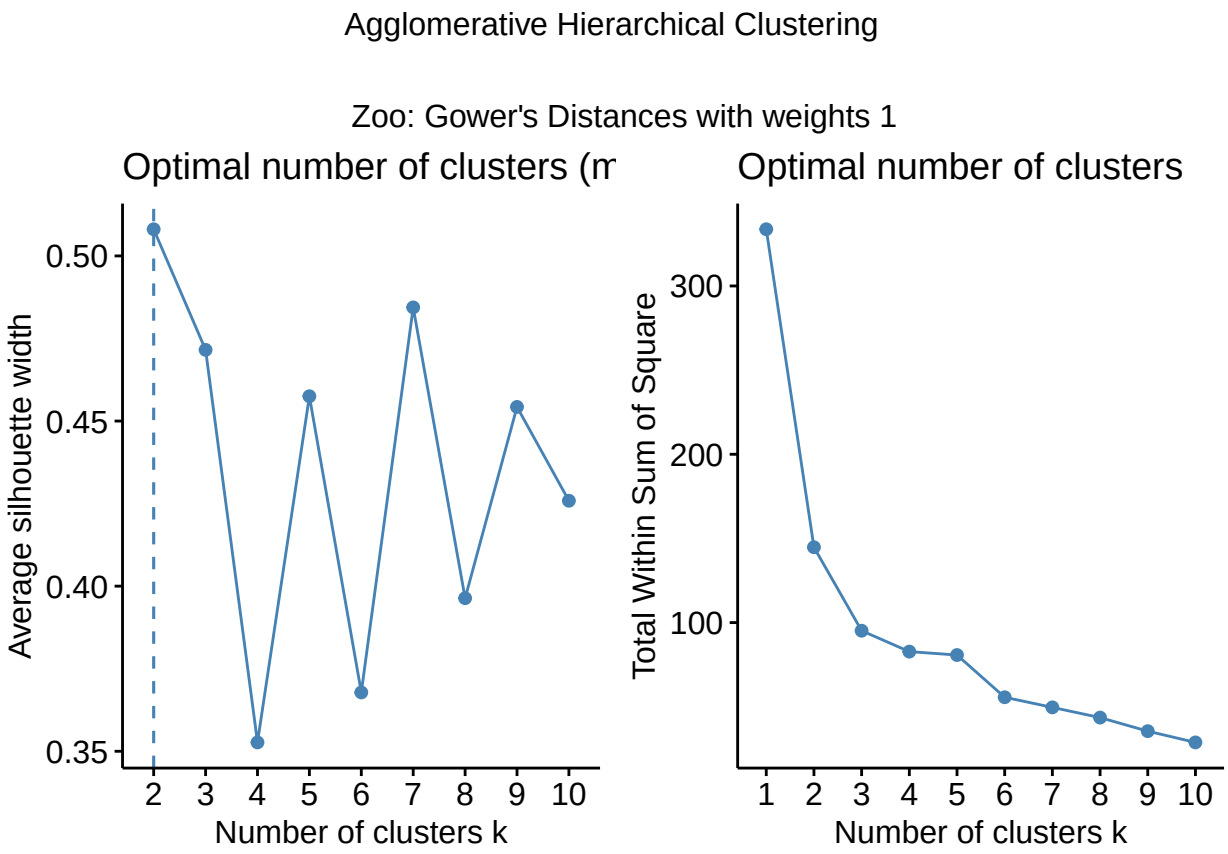
Table 31: Zoo: Methods performance

method_name	variables	parameters	rand.seed	homo_score
bagged k-modes	nominal	k=4,iter=100	42	0.71

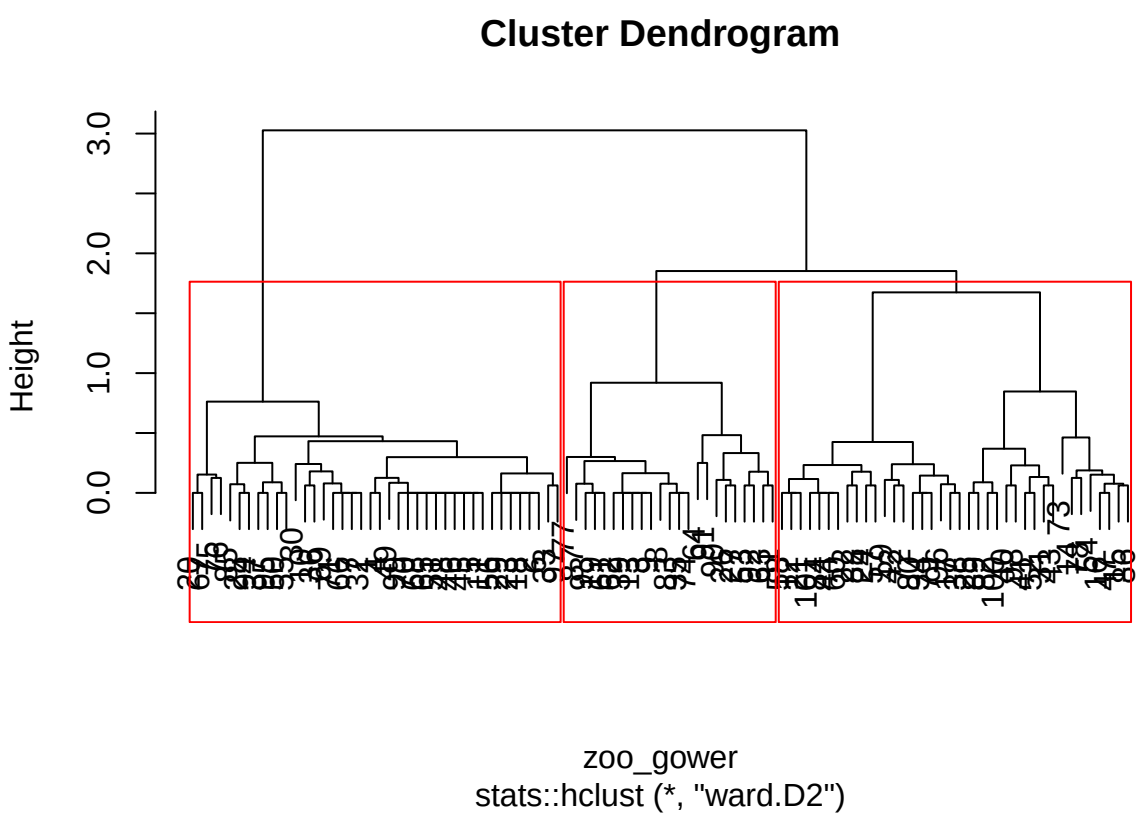
Zoo: HC

```
zoo_gower <- cluster::daisy(zoo_features,metric = "gower")
set.seed(42)
```

```
grid.arrange(
  factoextra::fviz_nbclust(as.matrix(zoo_gower), kmeans, method = "silhouette"),
  factoextra::fviz_nbclust(as.matrix(zoo_gower), kmeans, method = "wss"),
  ncol = 2,
  top = "Agglomerative Hierarchical Clustering \n
  Zoo: Gower's Distances with weights 1"
  )
```



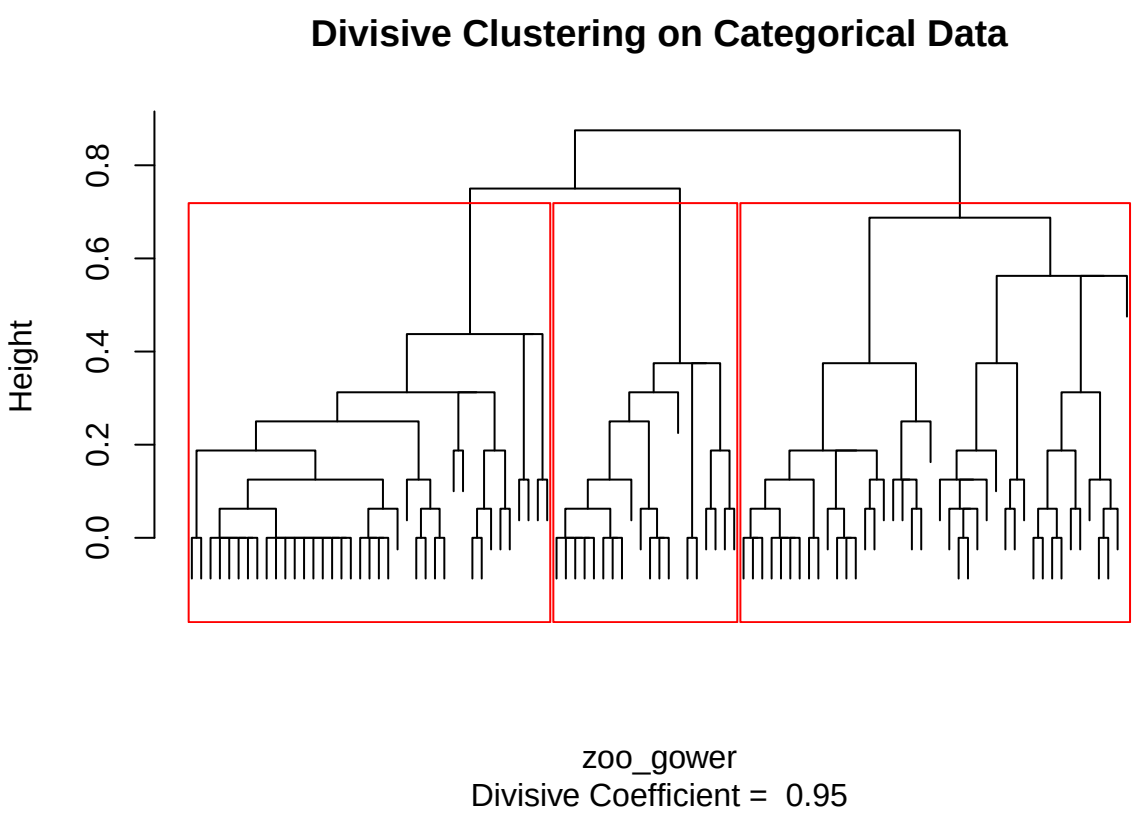

```
set.seed(42)
zoo_hc <- stats::hclust(zoo_gower, method = "ward.D2")
plot(zoo_hc)
rect.hclust(zoo_hc, k = 3, border = "red")
```




```
zoo_hc_classes <- stats::cutree(zoo_hc, k = 3)
```

This suggests 2 or 3 clusters, we go with 3.

```
set.seed(42)
zoo_hc_div <- cluster::diana(zoo_gower)
plot(
  zoo_hc_div, which.plots = 2, labels=FALSE,
  main = "Divisive Clustering on Categorical Data")
rect.hclust(as.hclust(zoo_hc_div), k = 3, border = "red")
```




```
zoo_hc_div_classes <- stats::cutree(as.hclust(zoo_hc_div), k = 3)
```

Table 32: Zoo: Predicted clusters HC-DIV k=3

	1	2	3
mammal	39	2	0
bird	0	0	20
reptile	0	4	1
fish	0	13	0
amphibian	0	1	3
insect	0	0	8
mollusc.et.al	0	0	10

Table 33: Zoo: Predicted clusters HC-AGG k=3

	1	2	3
mammal	40	1	0
bird	0	0	20
reptile	0	5	0
fish	0	13	0
amphibian	0	4	0
insect	0	0	8
mollusc.et.al	0	0	10

Table 34: Zoo: Methods performance

method_name	variables	parameters	rand.seed	homo_score
bagged k-modes	nominal	k=4,iter=100	42	0.71
HC-AGG	nominal	k=3	42	0.62
HC-DIV	nominal	k=3	42	0.56

Zoo: LCM

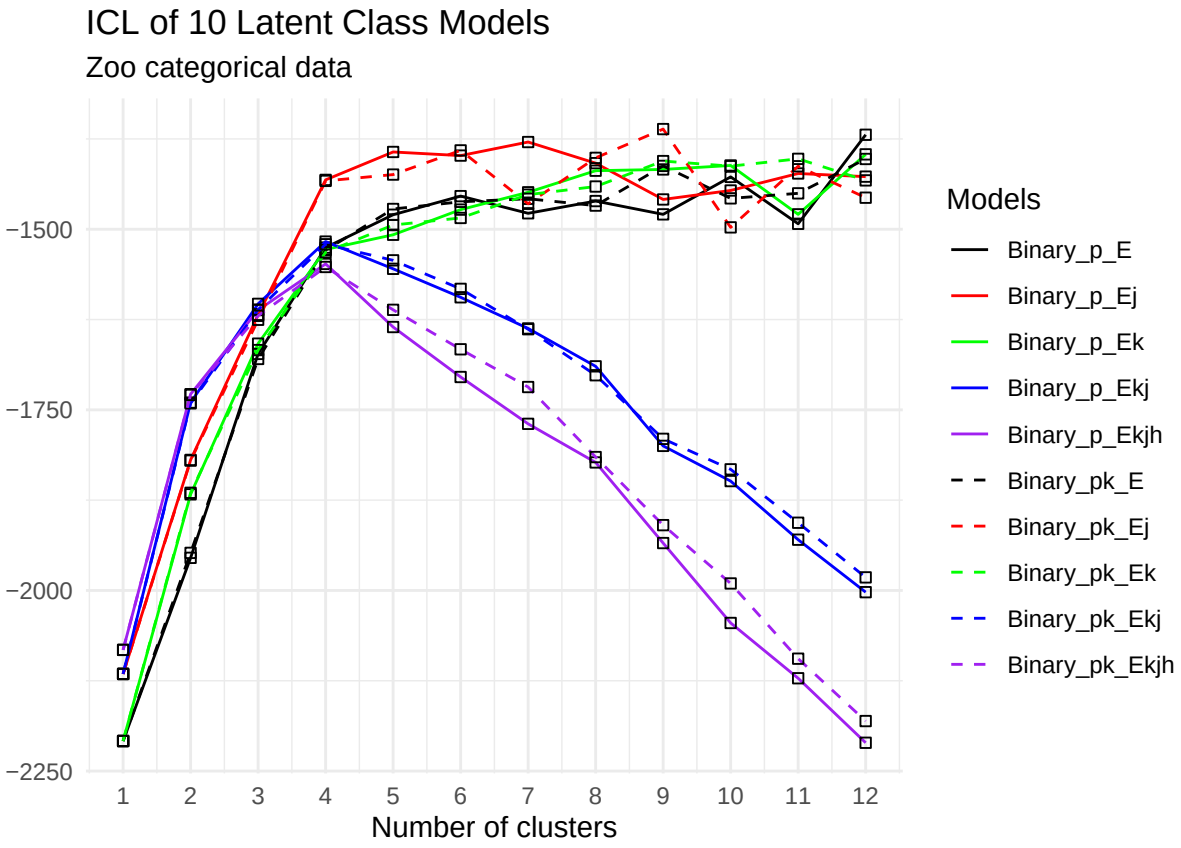


Table 35: Zoo: Predicted clusters LCM Binary_pk_Ej

	1	2	3	4	5	6	7	8	9
mammal	0	0	4	6	0	0	0	0	31
bird	16	0	0	0	4	0	0	0	0
reptile	0	1	0	0	0	0	4	0	0
fish	0	13	0	0	0	0	0	0	0
amphibian	0	0	0	0	0	0	4	0	0
insect	0	0	0	0	0	8	0	0	0
mollusc.et.al	0	0	0	0	0	3	0	7	0

Table 36: Zoo: Methods performance

method_name	variables	parameters	rand.seed	homo_score
LCM	nominal	k=11,Binary_pk_Ekjh	42	0.82
HC-AGG	nominal	k=8,FAST,100 Modes	42	0.76
HC-DIV	nominal	k=8,FAST,100 Modes	42	0.67
HC-AGG	nominal	k=5,FAST,100 Modes	42	0.62
HC-DIV	nominal	k=5,FAST,100 Modes	42	0.59
bagged k-modes	nominal	k=7,boot=100	42	0.56

The LCM outperforms.

References

C. Bouveyron, G. Celeux, T. Murphy, and A. Raftery. *Model-based Clustering and Classification for Data Science*. Birmingham, United Kingdom: Packt, 2019.

Richard Forsyth. Zoo Dataset. UCI Machine Learning Repository, 1990. URL https://archive.ics.uci.edu/ml/datasets/zoo. Dataset.

Brett Lantz. *Machine Learning with R*. Packt Publishing, 2013. ISBN 978-1784394523. Demonstrates rule-induction modeling where odor serves as the foundational root classifier for mushroom edibility.

A. Malik and B. Tuckfield. *Applied Unsupervised Learning with R*. Birmingham, United Kingdom: Packt, 2019.